PHP

Introduction

- PHP is a server scripting language, and a powerful tool for making dynamic and interactive Web pages.
- PHP is a widely-used, free, and efficient alternative to competitors such as Microsoft's ASP.

19.1 Introduction

- PHP, or PHP: Hypertext Preprocessor, has become the most popular server-side scripting language for creating dynamic web pages.
- PHP is open source and platform independent—implementations exist for all major UNIX, Linux, Mac and Windows operating systems. PHP also supports a large number of databases.

What is a PHP File?

- PHP files can contain text, HTML, CSS, JavaScript, and PHP code
- PHP code are executed on the server, and the result is returned to the browser as plain HTML
- PHP files have extension ".php"

What Can PHP Do?

- PHP can generate dynamic page content
- PHP can create, open, read, write, delete, and close files on the server
- PHP can collect data
- PHP can send and receive cookies
- PHP can add, delete, modify data in your database
- PHP can be used to control user-access
- PHP can encrypt data

With PHP you are not limited to output HTML. You can output images, PDF files, and even Flash movies. You can also output any text, such as XHTML and XML.

Why PHP? **

- PHP runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)
- PHP is compatible with almost all servers used today (Apache, IIS, Mercury, FileZilla, Tomcat etc.)
- PHP supports a wide range of databases
- PHP is free. Download it from the official PHP resource: www.php.net
- PHP is easy to learn and runs efficiently on the server side

19.2 A Simple PHP Program

- The power of the web resides not only in serving content to users, but also in responding to requests from users and generating web pages with dynamic content.
- PHP code is embedded directly into text-based documents, such as HTML, though these script segments are interpreted by a server *before* being delivered to the client.
- PHP script file names end with `.php`.
- In PHP, code is inserted between the scripting delimiters `<?php` and `?>`. PHP code can be placed anywhere in HTML5 markup, as long as the code is enclosed in these delimiters.

19.2 A Simple PHP Program (Cont.)

- Variables are preceded by a `$` and are created the first time they're encountered.
- PHP statements terminate with a semicolon (`;`).
- Single-line comments which begin with two forward slashes (`//`) or a pound sign (#). Text to the right of the delimiter is ignored by the interpreter. Multiline comments begin with delimiter `/*` and end with delimiter `*/`.
- When a variable is encountered inside a double-quoted (`""`) string, PHP interpolates the variable. In other words, PHP inserts the variable's value where the variable name appears in the string.
- All operations requiring PHP interpolation execute on the server before the HTML5 document is sent to the client.
- PHP variables are loosely typed—they can contain different types of data at different times.

PHP Data Types

Type	Description
`int`, `integer`	Whole numbers (i.e., numbers without a decimal point).
`float`, `double`, `real`	Real numbers (i.e., numbers containing a decimal point).
`string`	Text enclosed in either single (`''`) or double (`""`) quotes. [*Note:* Using double quotes allows PHP to recognize more escape sequences.]
`bool`, `boolean`	`true` or `false`.
`array`	Group of elements.
`object`	Group of associated data and methods.
`resource`	An external source—usually information from a database.
`NULL`	No value.

Fig. 19.2 | PHP types.

PHP 5 Syntax

- A PHP script is executed on the server, and the plain HTML result is sent back to the browser.
- A PHP script can be placed anywhere in the document.
- A PHP script starts with **<?php** and ends with **?>**:
- A PHP file normally contains HTML tags, and some PHP scripting code.
- PHP statements end with a semicolon (;).

PHP Case Sensitivity

- In PHP, all keywords (e.g. if, else, while, echo, etc.), classes, functions, and user-defined functions are **NOT case-sensitive**.
- All variable names are **case-sensitive**.

```php
<?php
ECHO "Hello World!<br>";
echo "Hello World!<br>";
EcHo "Hello World!<br>";
?>
```

```php
<?php
$color = "red";
echo "My car is " . $color . "<br>";
echo "My house is " . $COLOR . "<br>";
echo "My boat is " . $coLOR . "<br>";
?>
```

PHP Variables

- A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).

Rules for PHP variables:

- A variable starts with the $ sign, followed by the name of the variable
- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive ($age and $AGE are two different variables)

PHP is a Loosely Typed Language

- We did not have to tell PHP which data type the variable is.
- PHP automatically converts the variable to the correct data type, depending on its value.
- In other languages such as C, C++, and Java, the programmer must declare the name and type of the variable before using it.

PHP Variables Scope

- In PHP, variables can be declared anywhere in the script.
- The scope of a variable is the part of the script where the variable can be referenced/used.
- PHP has three different variable scopes:
  - local
  - global
  - static

Global and Local Scope

- A variable declared **outside** a function has a GLOBAL SCOPE and can only be accessed outside a function:

```php
<?php
$x = 5; // global scope

function myTest() {
    // using x inside this function will generate an error
    echo "<p>Variable x inside function is: $x</p>";
}
myTest();

echo "<p>Variable x outside function is: $x</p>";
?>
```

Global and Local Scope

- A variable declared **within** a function has a LOCAL SCOPE and can only be accessed within that function:

```php
<?php
function myTest() {
    $x = 5; // local scope
    echo "<p>Variable x inside function is: $x</p>";
}
myTest();

// using x outside the function will generate an error
echo "<p>Variable x outside function is: $x</p>";
?>
```

PHP Data Types

- Variables can store data of different types, and different data types can do different things.
- PHP supports the following data types:
  - String
  - Integer
  - Float (floating point numbers - also called double)
  - Boolean
  - Array
  - Object
  - NULL
  - Resource

PHP String

- A string is a sequence of characters, like "Hello world!".
- A string can be any text inside quotes. You can use single or double quotes:

```php
<?php
$x = "Hello world!";
$y = 'Hello world!';

echo $x;
echo "<br>";
echo $y;
?>
```

PHP String

- The PHP strlen() function returns the length of a string.
- The PHP str_word_count() function counts the number of words in a string:
- The PHP strrev() function reverses a string:

```php
<?php
echo strlen("Hello world!"); // outputs 12
?>
```

```php
<?php
echo strrev("Hello world!"); // outputs !dlrow olleH
?>
```

```php
<?php
echo str_word_count("Hello world!"); // outputs 2
?>
```

PHP String(Search For a Specific Text Within a String**)**

- The PHP strpos() function searches for a specific text within a string.
- If a match is found, the function returns the character position of the first match. If no match is found, it will return FALSE.
- The example below searches for the text "world" in the string "Hello world!":

```php
<?php
echo strpos("Hello world!", "world"); // outputs 6
?>
```

Replace Text Within a String

- The PHP str_replace() function replaces some characters with some other characters in a string.
- The example below replaces the text "world" with "Dolly":

```php
<?php
echo str_replace("world", "Dolly", "Hello world!"); // outputs Hello Dolly!
?>
```

PHP Integer

- An integer data type is a non-decimal number between -2,147,483,648 and 2,147,483,647.
- **Rules for integers:**
  - An integer must have at least one digit
  - An integer must not have a decimal point
  - An integer can be either positive or negative
  - Integers can be specified in three formats: decimal (10-based), hexadecimal (16-based - prefixed with 0x) or octal (8-based - prefixed with 0)

PHP Integer

- In the following example $x is an integer. The PHP var_dump() function returns the data type and value:

```
<!DOCTYPE html>
<html>
<body>

<?php
$x = 5985;
var_dump($x);
?>

</body>
</html>
```

Result:

int(5985)

PHP Data Types

- A float (floating point number) is a number with a decimal point or a number in exponential form.
- A Boolean represents two possible states: TRUE or FALSE.
- An array stores multiple values in one single variable.
- Null is a special data type which can have only one value: NULL.

PHP Object

- An object is a data type which stores data and information on how to process that data.
- In PHP, an object must be explicitly declared.
- First we must declare a class of object. For this, we use the class keyword. A class is a structure that can contain properties and methods:

```php
<?php
class Car {
    function Car() {
        $this->model = "VW";
    }
}

// create an object
$herbie = new Car();

// show object properties
echo $herbie->model;
?>
```

PHP Resource

- The special resource type is not an actual data type. It is the storing of a reference to functions and resources external to PHP.
- A common example of using the resource data type is a database call.

PHP Constants

- A constant is an identifier (name) for a simple value.
- The value cannot be changed during the script.
- A valid constant name starts with a letter or underscore (no $ sign before the constant name).
- To create a constant, use the define() function.
- Constants are automatically global and can be used across the entire script.

PHP Constants

- **Syntax**
  - define(*name*, *value*, *case-insensitive*)
- **Parameters:**
  - *name*: Specifies the name of the constant
  - *value*: Specifies the value of the constant
  - *case-insensitive*: Specifies whether the constant name should be case-insensitive. Default is false

PHP Constants

- The example below creates a constant with a **case-sensitive** name:

```php
<?php
define("GREETING", "Welcome to W3Schools.com!");
echo GREETING;
?>
```

- The example below creates a constant with a **case-insensitive** name:

```php
<?php
define("GREETING", "Welcome to W3Schools.com!", true);
echo greeting;
?>
```

PHP Operators

- Operators are used to perform operations on variables and values.
- PHP divides the operators in the following groups:
  - Arithmetic operators
  - Assignment operators
  - Comparison operators
  - Increment/Decrement operators
  - Logical operators
  - String operators
  - Array operators

PHP Arithmetic Operators

- The PHP arithmetic operators are used with numeric values to perform common arithmetical operations, such as addition, subtraction, multiplication etc.

Operator	Name	Example	Result
+	Addition	$x + $y	Sum of $x and $y
-	Subtraction	$x - $y	Difference of $x and $y
*	Multiplication	$x * $y	Product of $x and $y
/	Division	$x / $y	Quotient of $x and $y
%	Modulus	$x % $y	Remainder of $x divided by $y

PHP Assignment Operators

- The PHP assignment operators are used with numeric values to write a value to a variable.
- The basic assignment operator in PHP is "=". It means that the left operand gets set to the value of the assignment expression on the right.

Assignment	Same as...	Description
x = y	x = y	The left operand gets set to the value right
x += y	x = x + y	Addition
x -= y	x = x - y	Subtraction
x *= y	x = x * y	Multiplication
x /= y	x = x / y	Division
x %= y	x = x % y	Modulus

PHP Conditional Statements

- Very often when you write code, you want to perform different actions for different conditions. You can use conditional statements in your code to do this.
- In PHP we have the following conditional statements:
  - **if statement** - executes some code if one condition is true
  - **if...else statement** - executes some code if a condition is true and another code if that condition is false
  - **if...elseif....else statement** - executes different codes for more than two conditions
  - **switch statement** - selects one of many blocks of code to be executed

The PHP switch Statement

- Use the switch statement to **select one of many blocks of code to be executed**.

```
switch (n) {
    case label1:
        code to be executed if n=label1;
        break;
    case label2:
        code to be executed if n=label2;
        break;
    case label3:
        code to be executed if n=label3;
        break;
    ...
    default:
        code to be executed if n is different from all labels;
}
```

```
<?php
$favcolor = "red";

switch ($favcolor) {
    case "red":
        echo "Your favorite color is red!";
        break;
    case "blue":
        echo "Your favorite color is blue!";
        break;
    case "green":
        echo "Your favorite color is green!";
        break;
    default:
        echo "Your favorite color is neither red, blue, nor green!";
}
?>
```

PHP Loops

- Often when you write code, you want the same block of code to run over and over again in a row. Instead of adding several almost equal code-lines in a script, we can use loops to perform a task like this.
- In PHP, we have the following looping statements:
  - **while** - loops through a block of code as long as the specified condition is true
  - **do...while** - loops through a block of code once, and then repeats the loop as long as the specified condition is true
  - **for** - loops through a block of code a specified number of times
  - **foreach** - loops through a block of code for each element in an array

The PHP while Loop

- The while loop executes a block of code as long as the specified condition is true.

```
while (condition is true) {
    code to be executed;
}
```

```php
<?php
$x = 1;

while($x <= 5) {
  echo "The number is: $x <br>";
  $x++;
}
?>
```

The PHP do...while Loop

- The do...while loop will always execute the block of code once, it will then check the condition, and repeat the loop while the specified condition is true.

```
do {
    code to be executed;
} while (condition is true);
```

```
<?php
$x = 1;

do {
    echo "The number is: $x <br>";
    $x++;
} while ($x <= 5);
?>
```

```
<?php
$x = 6;

do {
    echo "The number is: $x <br>";
    $x++;
} while ($x <= 5);
?>
```

PHP Functions

- The real power of PHP comes from its functions; it has more than 1000 built-in functions.
- Besides the built-in PHP functions, we can create our own functions.
- A function is a block of statements that can be used repeatedly in a program.
- A function will not execute immediately when a page loads.
- A function will be executed by a call to the function.

```
function functionName() {
    code to be executed;
}
```

PHP Functions

- A function name can start with a letter or underscore (not a number).
- Give the function a name that reflects what the function does!
- Function names are NOT case-sensitive.

```php
<?php
function writeMsg() {
    echo "Hello world!";
}

writeMsg(); // call the function
?>
```

PHP Function Arguments

- Information can be passed to functions through arguments. An argument is just like a variable.
- Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

```
<!DOCTYPE html>
<html>
<body>

<?php
function familyName($fname) {
    echo "$fname Refsnes.<br>";
}

familyName("Jani");
familyName("Hege");
familyName("Stale");
familyName("Kai Jim");
familyName("Borge");
?>

</body>
</html>
```

Result:

Jani Refsnes.
Hege Refsnes.
Stale Refsnes.
Kai Jim Refsnes.
Borge Refsnes.


Thank you!
Any Questions ?

PHP-II

PHP - Dealing with the Client

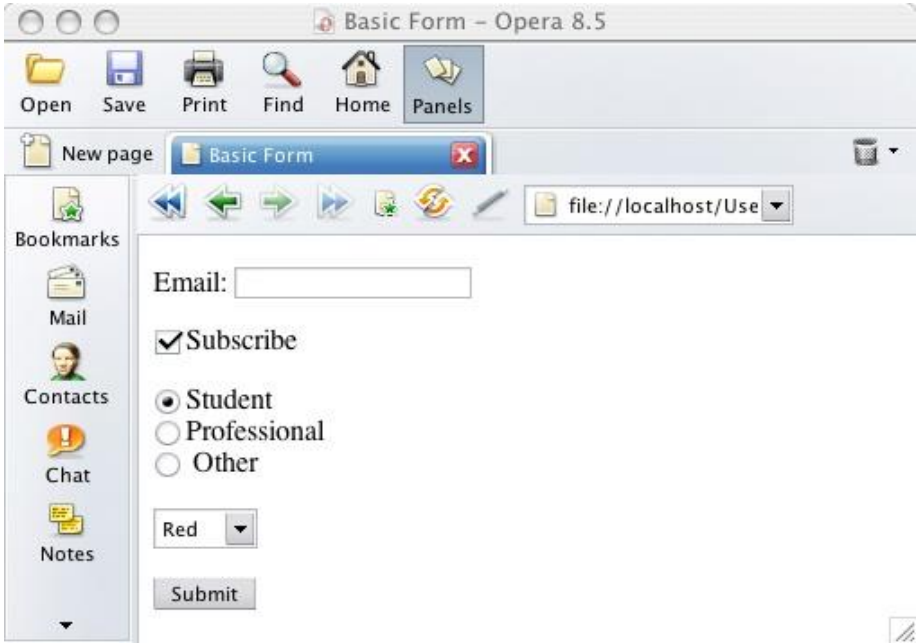
- All very nice but …
- … How is it useful in your web site?
- PHP allows you to use HTML forms
- Forms require technology at the server to process them
- PHP is a feasible and good choice for the processing of HTML forms

PHP - Dealing with the client

- Quick re-cap on forms
- Implemented with a <form> element in HTML
- Contains other input, text area, list controls and options
- Has some method of submitting

PHP - Dealing with the client

- Text fields
- Checkbox
- Radio button
- List boxes
- Hidden form fields
- Password box
- Submit and reset buttons



PHP - Dealing with the client

- <form method="post" action="file.php" name="frmid" >
  - Method specifies how the data will be sent
  - Action specifies the file to go to. E.g. file.php
  - id gives the form a unique name
- Post method sends all contents of a form with basically hidden headers (not easily visible to users)
- Get method sends all form input in the URL requested using name=value pairs separated by ampersands (&)
  - E.g. process.php?name=trevor&number=345
  - Is visible in the URL shown in the browser

PHP - Dealing with the client

- All form values are placed into an array
- Assume a form contains one textbox called “txtName” and the form is submitted using the post method, invoking process.php
- process.php could access the form data using:
  - $_POST[‘txtName’]
- If the form used the get method, the form data would be available as:
  - $_GET[‘txtName’]

PHP - Dealing with the client

- For example, an HTML form:
  - <form id="showmsg" action="show.php" method="post">
    - <input type="text" id="txtMsg" value="Hello World" />
    - <input type="submit" id="submit" value="Submit">
  - </form>

GET vs. POST *****

- Both GET and POST create an array (e.g. array(key => value, key2 => value2, key3 => value3, ...)).
- This array holds key/value pairs, where
  - **keys are the names of the form controls**
  - **values are the input data from the user.**
- Both GET and POST are treated as $_GET and $_POST.

GET vs. POST

- $_GET is an array of variables passed to the current script via the URL parameters.
- $_POST is an array of variables passed to the current script via the HTTP POST method.
- **These are superglobals, which means**
  - They are always accessible, regardless of scope .
  - You can access them from any function, class or file without having to do anything special.

When to use GET?

- Information sent from a form with the GET method is **visible to everyone** (all variable names and values are displayed in the URL).
- GET also has limits on the amount of information to send.
- The limitation is about 2000 characters.

When to use GET?

- However, because the variables are displayed in the URL, it is possible to bookmark the page.
- GET may be used for sending non-sensitive data.
- GET should NEVER be used for sending passwords or other sensitive information!

When to use POST?

- Information sent from a form with the POST method is **invisible to others**
- all names/values are embedded within the body of the HTTP request and
- has **no limits** on the amount of information to send.

When to use POST?

- Moreover POST supports advanced functionality such as support for multi-part binary input while uploading files to server.
- However, because the variables are not displayed in the URL, it is not possible to bookmark the page.
- **Developers prefer POST for sending form data.**

Using HTML Forms

```php
<?php
   if( $_POST["name"] || $_POST["age"] ) {
      if (preg_match("/[^A-Za-z'-]/",$_POST['name'] )) {
         die ("invalid name and name should be alpha");
      }

      echo "Welcome ". $_POST['name']. "<br />";
      echo "You are ". $_POST['age']. " years old.";

      exit();
   }
?>
<html>
   <body>

      <form action = "<?php $_PHP_SELF ?>" method = "POST">
         Name: <input type = "text" name = "name" />
         Age: <input type = "text" name = "age" />
         <input type = "submit" />
      </form>

   </body>
</html>
```

It will produce the following result –

PHP Arrays

- > An array variable is a storage area holding a number or text. The problem is, a variable will hold only one value.
- > An array is a special variable, which can store multiple values in one single variable.

PHP Arrays

- If you have a list of items (a list of car names, for example), storing the cars in single variables could look like this:

```
$cars1="Saab";
$cars2="Volvo";
$cars3="BMW";
```

PHP Arrays

- > However, what if you want to loop through the cars and find a specific one? And what if you had not 3 cars, but 300?
- > The best solution here is to use an array.
- > An array can hold all your variable values under a single name. And you can access the values by referring to the array name.
- > Each element in the array has its own index so that it can be easily accessed.

PHP Arrays

In PHP, there are three kind of arrays:

- **> Numeric array** - An array with a numeric index
- **> Associative array** - An array where each ID key is associated with a value
- **> Multidimensional array** - An array containing one or more arrays

PHP Numeric Arrays

- > A numeric array stores each array element with a numeric index.
- > There are two methods to create a numeric array.

PHP Numeric Arrays

- In the following example the index is automatically assigned (the index starts at 0):

```
$cars=array("Saab","Volvo","BMW","Toyota");
```

- In the following example we assign the index manually:

```
$cars[0]="Saab";
$cars[1]="Volvo";
$cars[2]="BMW";
$cars[3]="Toyota";
```

PHP Numeric Arrays

- In the following example you access the variable values by referring to the array name and index:

```
<?php
$cars[0]="Saab";
$cars[1]="Volvo";
$cars[2]="BMW";
$cars[3]="Toyota";
echo $cars[0] . " and " . $cars[1] . " are Swedish cars.";
?>
```

```
Saab and Volvo are Swedish cars.
```

- The code above will output:

PHP Associative Arrays

- > With an associative array, each ID key is associated with a value.
- > When storing data about specific named values, a numerical array is not always the best way to do it.
- > With associative arrays we can use the values as keys and assign values to them.

PHP Associative Arrays

- In this example we use an array to assign ages to the different persons:

```
$ages = array("Peter"=>32, "Quagmire"=>30, "Joe"=>34);
```

- This example is the same as the one above, but shows a different way of creating the array:

```
$ages['Peter'] = "32";
$ages['Quagmire'] = "30";
$ages['Joe'] = "34";
```

PHP Associative Arrays

The ID keys can be used in a script:

```
<?php
$ages['Peter'] = "32";
$ages['Quagmire'] = "30";
$ages['Joe'] = "34";

echo "Peter is " . $ages['Peter'] . " years old.";
?>
```

The code above will output:

```
Peter is 32 years old.
```

PHP Multidimensional Arrays

- In a multidimensional array, each element in the main array can also be an array.
  - For a two-dimensional array you need two indices to select an element
  - For a three-dimensional array you need three indices to select an element
- And each element in the sub-array can be an array, and so on.

PHP Multidimensional Arrays

```php
<?php
$cars = array
  (
  array("Volvo",22,18),
  array("BMW",15,13),
  array("Saab",5,2),
  array("Land Rover",17,15)
  );

echo $cars[0][0].": In stock: ".$cars[0][1].", sold: ".$cars[0][2].".<br>";
echo $cars[1][0].": In stock: ".$cars[1][1].", sold: ".$cars[1][2].".<br>";
echo $cars[2][0].": In stock: ".$cars[2][1].", sold: ".$cars[2][2].".<br>";
echo $cars[3][0].": In stock: ".$cars[3][1].", sold: ".$cars[3][2].".<br>";
?>
```

Result:

Volvo: In stock: 22, sold: 18.
BMW: In stock: 15, sold: 13.
Saab: In stock: 5, sold: 2.
Land Rover: In stock: 17, sold: 15.

PHP Multidimensional Arrays

Lets try displaying a single value from the array above:

```
echo "Is " . $families['Griffin'][2] .
" a part of the Griffin family?";
```

The code above will output:

```
Is Megan a part of the Griffin family?
```

Concatenation

- Use a period to join strings into one.

```
<?php
$string1="Hello";
$string2="PHP";
$string3=$string1 . " " . $string2;
Print $string3;
?>
```

```
Hello PHP
```

PHP include and require Statements

****:

- The include (or require) statement takes all the text/code/markup that exists in the specified file and copies it into the file that uses the include statement.
- Including files is very useful when you want to include the same PHP, HTML, or text on multiple pages of a website.

PHP include and require Statements

- **The include and require statements are identical, except upon failure:**
  - require will produce a fatal error (E_COMPILE_ERROR) and stop the script
  - include will only produce a warning (E_WARNING) and the script will continue
- Use **require** when the file is required by the application.
- Use **include** when the file is not required and application should continue when file is not found.

PHP include and require Statements

Assume we have a standard footer file called "footer.php", that looks like this:

```
<?php
echo "<p>Copyright &copy; 1999-" . date("Y") . " W3Schools.com</p>";
?>
```

```
<!DOCTYPE html>
<html>
<body>

<h1>Welcome to my home page!</h1>
<p>Some text.</p>
<p>Some more text.</p>
<?php include 'footer.php';?>

</body>
</html>
```

Result:

Welcome to my home page!

Some text.

Some more text.

Copyright © 1999-2016 W3Schools.com

What is a Cookie? *****

- A cookie is often used to identify a user.
- A cookie is a small file that the server embeds on the user's computer.
- Each time the same computer requests a page with a browser, it will send the cookie too.
- With PHP, you can both create and retrieve cookie values.

Create Cookies With PHP

- **A cookie is created with the setcookie() function.**

Syntax

```
setcookie(name, value, expire, path, domain, secure, httponly);
```

- Only the *name* parameter is required. All other parameters are optional.

Create Cookies With PHP

- PHP can
  - **Create/Retrieve a Cookie**
  - **Modify a Cookie Value**
  - **Delete a Cookie**
  - **Check if Cookies are Enabled**

- The **setcookie()** function must appear BEFORE the <html> tag.

What is a PHP Session? *****

- A session is a way to store information (in variables) to be used across multiple pages.
- Unlike a cookie, the information is not stored on the users computer.
- Session variables hold information about one single user, and are available to all pages in one application.

What is a PHP Session?

- When you work with an application, you open it, do some changes, and then you close it. This is much like a Session. The computer knows who you are. It knows when you start the application and when you end. But on the internet there is one problem: the web server does not know who you are or what you do, because the HTTP address doesn't maintain state.
- Session variables solve this problem by storing user information to be used across multiple pages (e.g. username, favorite color, etc). By default, session variables last until the user closes the browser.

Start a PHP Session

- A session is started with the session_start() function.
- Session variables are set with the PHP global variable: $_SESSION.

```php
<?php
// Start the session
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// Set session variables
$_SESSION["favcolor"] = "green";
$_SESSION["favanimal"] = "cat";
echo "Session variables are set.";
?>

</body>
</html>
```

How does it work?

- Most sessions set a user-key on the user's computer that looks something like this: 765487cf34ert8dede5a562e4f3a7e12.
- Then, when a session is opened on another page, it scans the computer for a user-key.
- If there is a match, it accesses that session, if not, it starts a new session.

PHP - Error & Exception Handling

- Error handling is the process of catching errors raised by your program and then taking appropriate action.
- If you would handle errors properly then it may lead to many unforeseen consequences.
- Its very simple in PHP to handle an errors.

PHP - Error & Exception Handling

- Lets explain there new keyword related to exceptions.
  - **Try** − A function using an exception should be in a "try" block. If the exception does not trigger, the code will continue as normal. However if the exception triggers, an exception is "thrown".
  - **Throw** − This is how you trigger an exception. Each "throw" must have at least one "catch".
  - **Catch** − A "catch" block retrieves an exception and creates an object containing the exception information.

PHP - Error & Exception Handling

- There are following functions which can be used from **Exception** class.
  - **getMessage()** – message of exception
  - **getCode()** – code of exception
  - **getFile()** – source filename
  - **getLine()** – source line
  - **getTrace()** – n array of the backtrace()
  - **getTraceAsString()** – formated string of trace

Exercise

Salary of Mr. A is	1000$
Salary of Mr. B is	1200$
Salary of Mr. C is	1400$

Exercise

Salary of Mr. A is	1000$
Salary of Mr. B is	1200$
Salary of Mr. C is	1400$

view plain copy to clipboard print ?

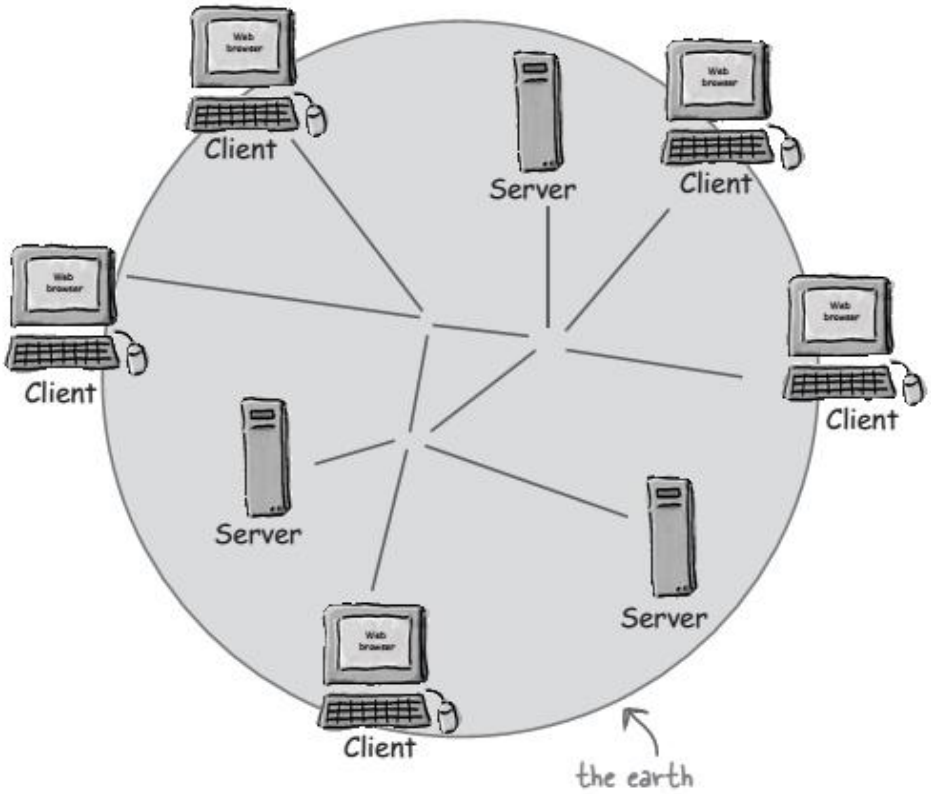
```
01. <?php
02. $a=1000;
03. $b=1200;
04. $c=1400;
05. echo "<table border=1 cellspacing=0 cellpading=0>
06. <tr> <td><font color=blue>Salary of Mr. A is</td> <td>$a$</font></td></tr>
07. <tr> <td><font color=blue>Salary of Mr. B is</td> <td>$b$</font></td></tr>
08. <tr> <td><font color=blue>Salary of Mr. C is</td> <td>$c$</font></td></tr>
09. </table>";
10. ?>
```

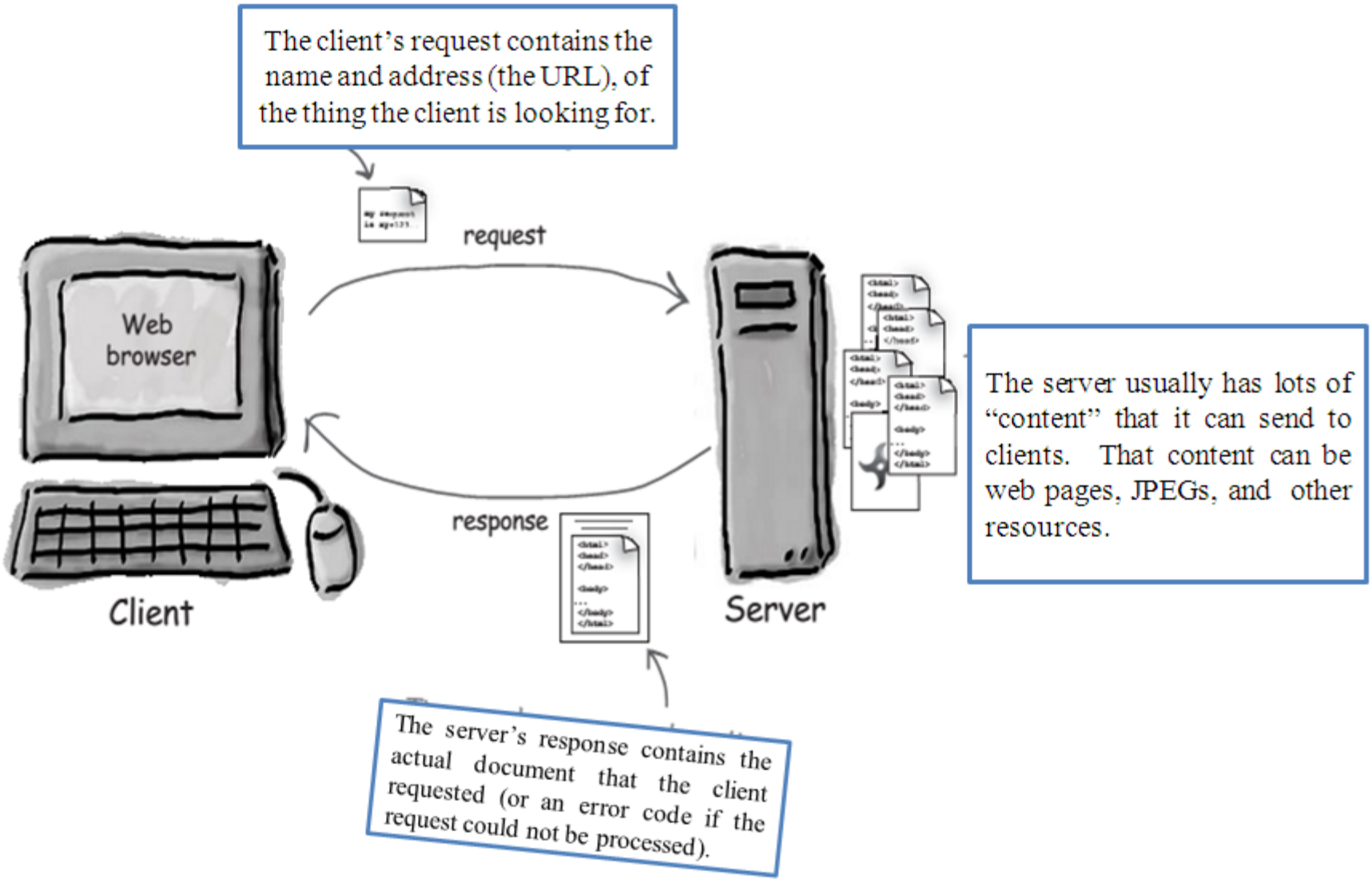

Thank you!
Any Questions ?

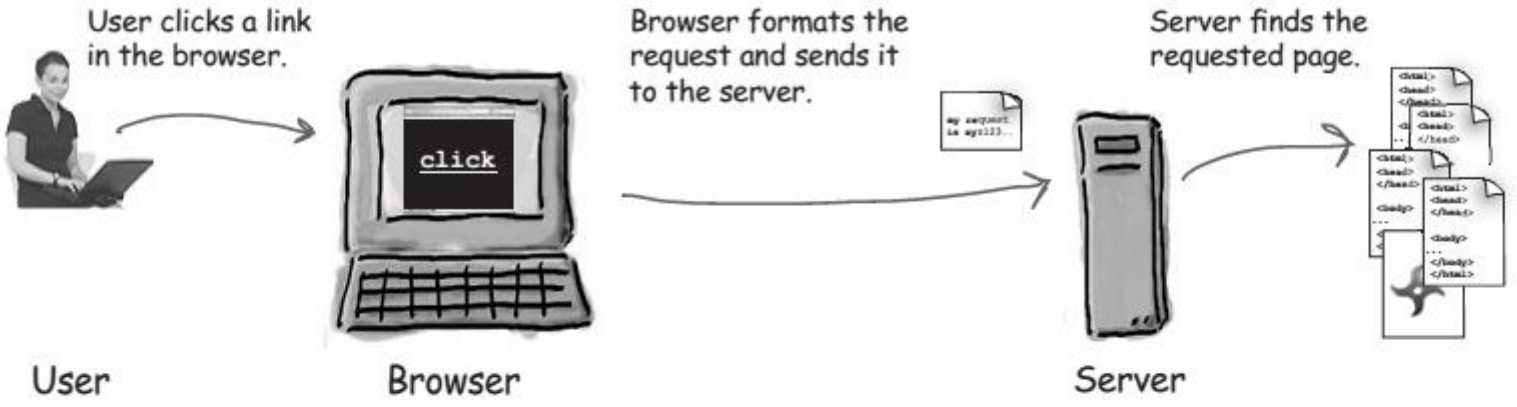
Web Engineering

HTTP Protocol

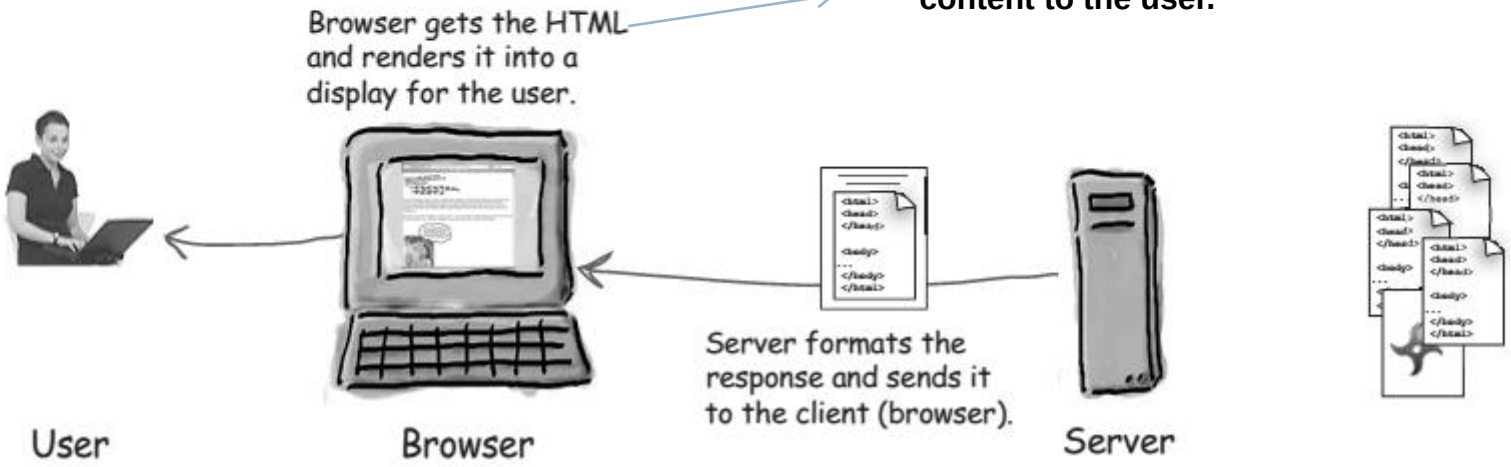
Internet and Web




The client's request contains the name and address (the URL), of the thing the client is looking for.
request
Web browser
Client
response
Server
The server usually has lots of "content" that it can send to clients. That content can be web pages, JPEGs, and other resources.
The server's response contains the actual document that the client requested (or an error code if the request could not be processed).



HTML tells the browser how to present the content to the user.



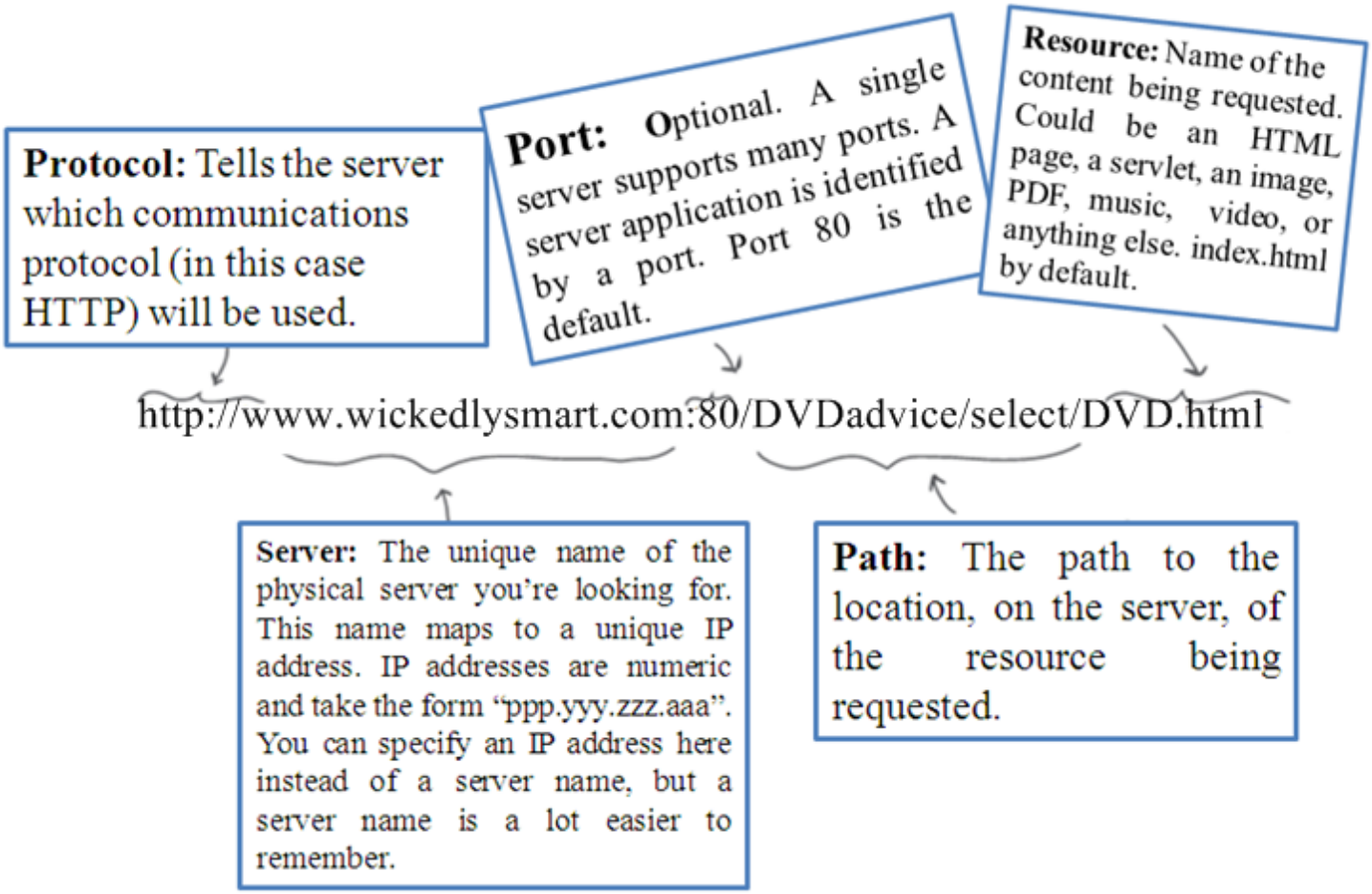
Web and HyperText Transfer Protocol (HTTP)

First some jargon

- Web page consists of objects
- Object can be HTML file, JPEG image, Java applet, audio file,…
- Web page consists of base HTML-file which includes several referenced objects
- Each object is addressable by a URL
- Example URL:

`www.someschool.edu/someDept/pic.gif`

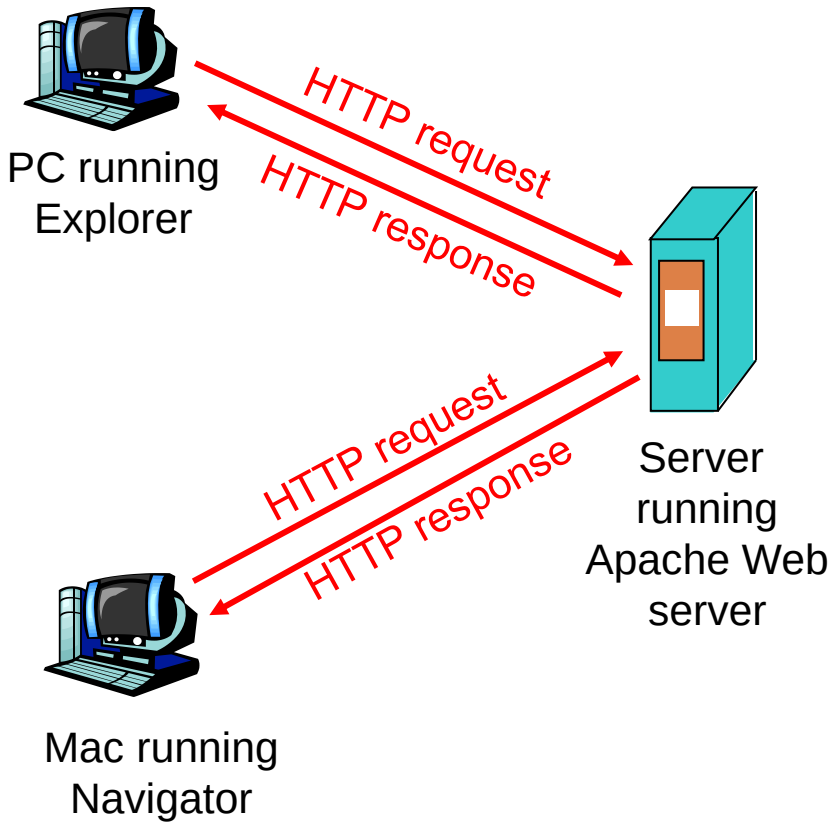
URL **


Protocol: Tells the server which communications protocol (in this case HTTP) will be used.
Port: Optional. A single server supports many ports. A server application is identified by a port. Port 80 is the default.
Resource: Name of the content being requested. Could be an HTML page, a servlet, an image, PDF, music, video, or anything else. index.html by default.
http://www.wickedlysmart.com:80/DVDadvice/select/DVD.html
Server: The unique name of the physical server you're looking for. This name maps to a unique IP address. IP addresses are numeric and take the form "ppp.yyy.zzz.aaa". You can specify an IP address here instead of a server name, but a server name is a lot easier to remember.
Path: The path to the location, on the server, of the resource being requested.

HTTP overview

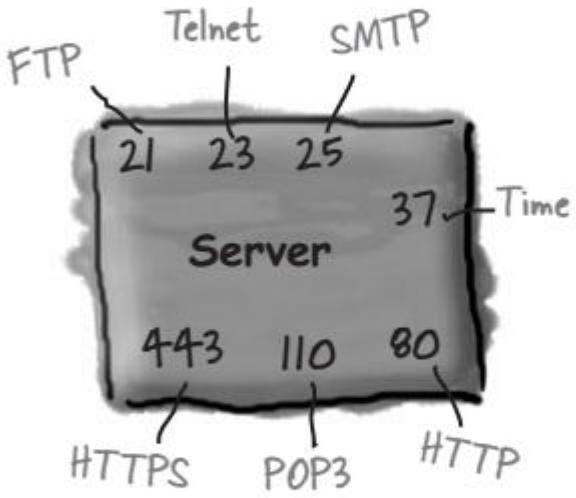
HTTP: hypertext transfer protocol

- Web's application layer protocol
- client/server model
  - *client:* browser that requests, receives, "displays" Web objects
  - *server:* Web server sends objects in response to requests
- HTTP 1.0: RFC 1945
- HTTP 1.1: RFC 2068



Ports

Well-known TCP port numbers for common server applications



Using one server app per port, a server can have up to 65536 different server apps running.

- The TCP port numbers from 0 to 1023 are reserved for well-known services.
- Don't use these ports for your own custom server programs!

HTTP overview (continued)

Uses TCP:

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

HTTP is "stateless"

- server maintains no information about past client requests

aside

Protocols that maintain "state" are complex!

- past history (state) must be maintained
- if server/client crashes, their views of "state" may be inconsistent, must be reconciled

HTTP connections

Nonpersistent HTTP

- At most one object is sent over a TCP connection.
- HTTP/1.0 uses nonpersistent HTTP

Persistent HTTP

- Multiple objects can be sent over single TCP connection between client and server.
- HTTP/1.1 uses persistent connections in default mode

Nonpersistent HTTP

Math differenc

Suppose user enters URL

`www.someSchool.edu/someDepartment/home.index`

(contains text, references to 10 jpeg images)

1a. HTTP client initiates TCP connection to HTTP server (process) at www.someSchool.edu on port 80

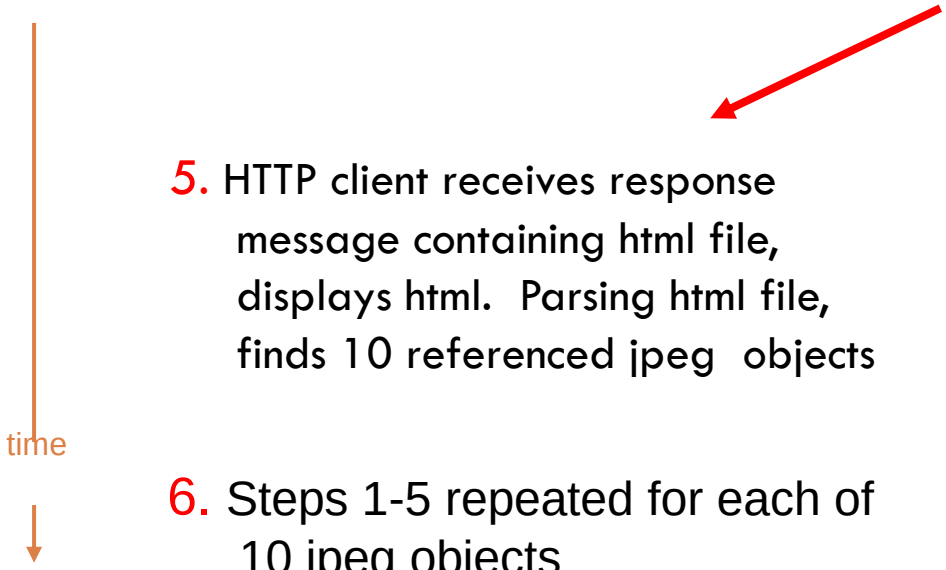
1b. HTTP server at host www.someSchool.edu waiting for TCP connection at port 80. "accepts" connection, notifying client

2. HTTP client sends HTTP *request message* (containing URL) into TCP connection socket. Message indicates that client wants object someDepartment/home.index

3. HTTP server receives request message, forms *response message* containing requested object, and sends message into its socket

time

Nonpersistent HTTP (cont.)



4. HTTP server closes TCP connection.

5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

6. Steps 1-5 repeated for each of 10 jpeg objects

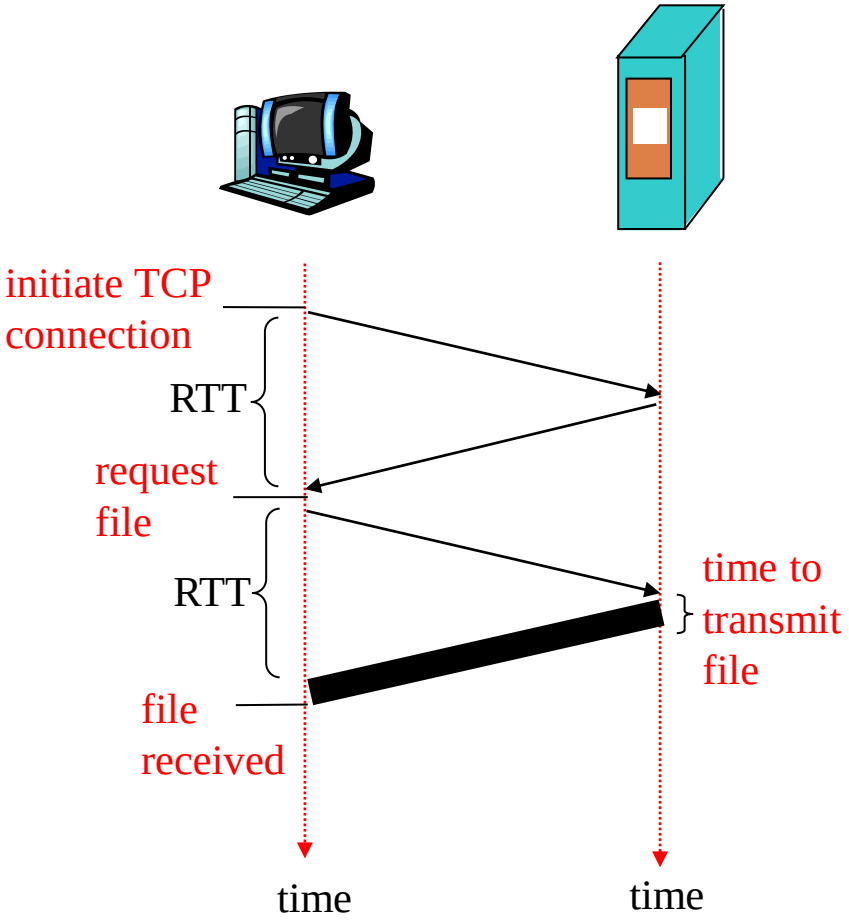
Response time modeling

Definition of RRT: time to send a small packet to travel from client to server and back.

Response time:

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- file transmission time

total = 2RTT+transmit time



Persistent HTTP

Nonpersistent HTTP issues:

- requires 2 RTTs per object
- OS must work and allocate host resources for each TCP connection
- but browsers often open parallel TCP connections to fetch referenced objects

Persistent HTTP

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server are sent over connection

Persistent without pipelining:

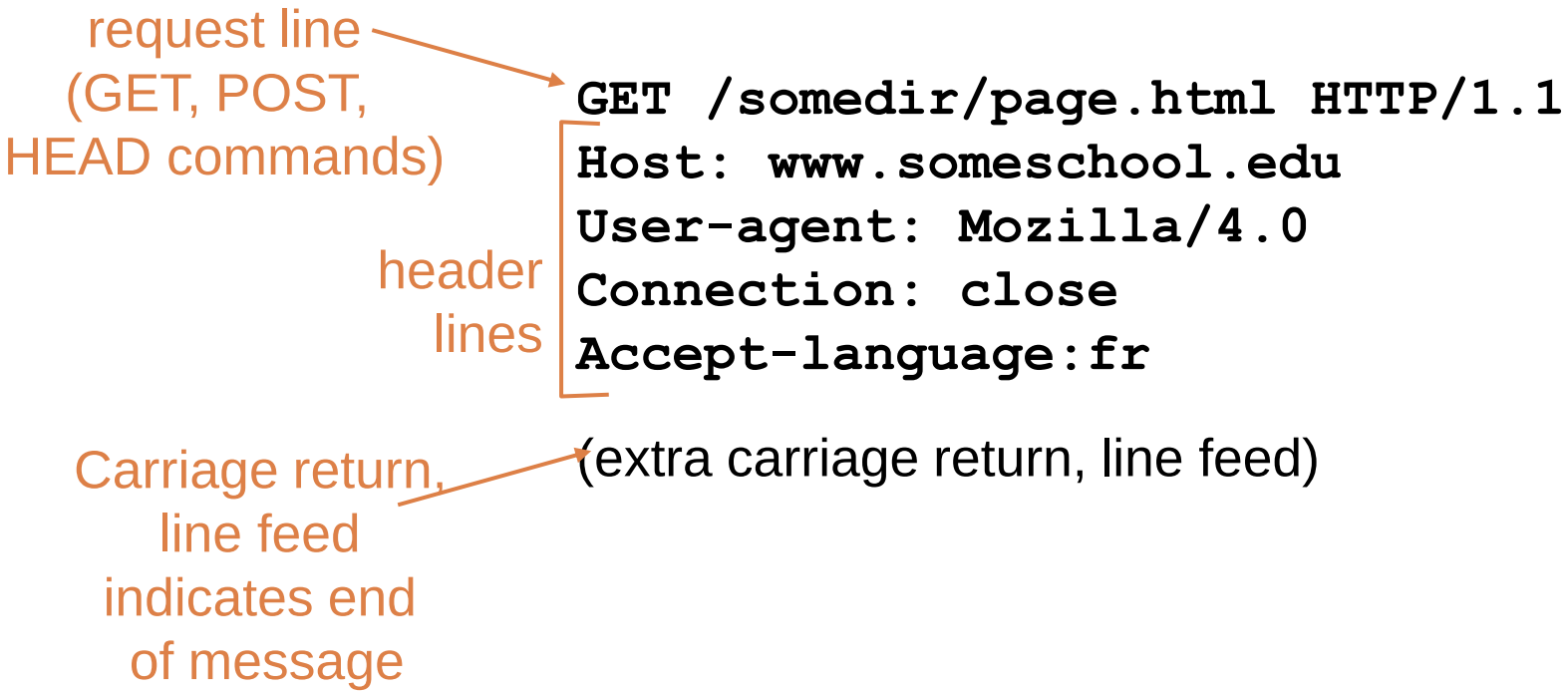
- client issues new request only when previous response has been received
- one RTT for each referenced object

Persistent with pipelining:

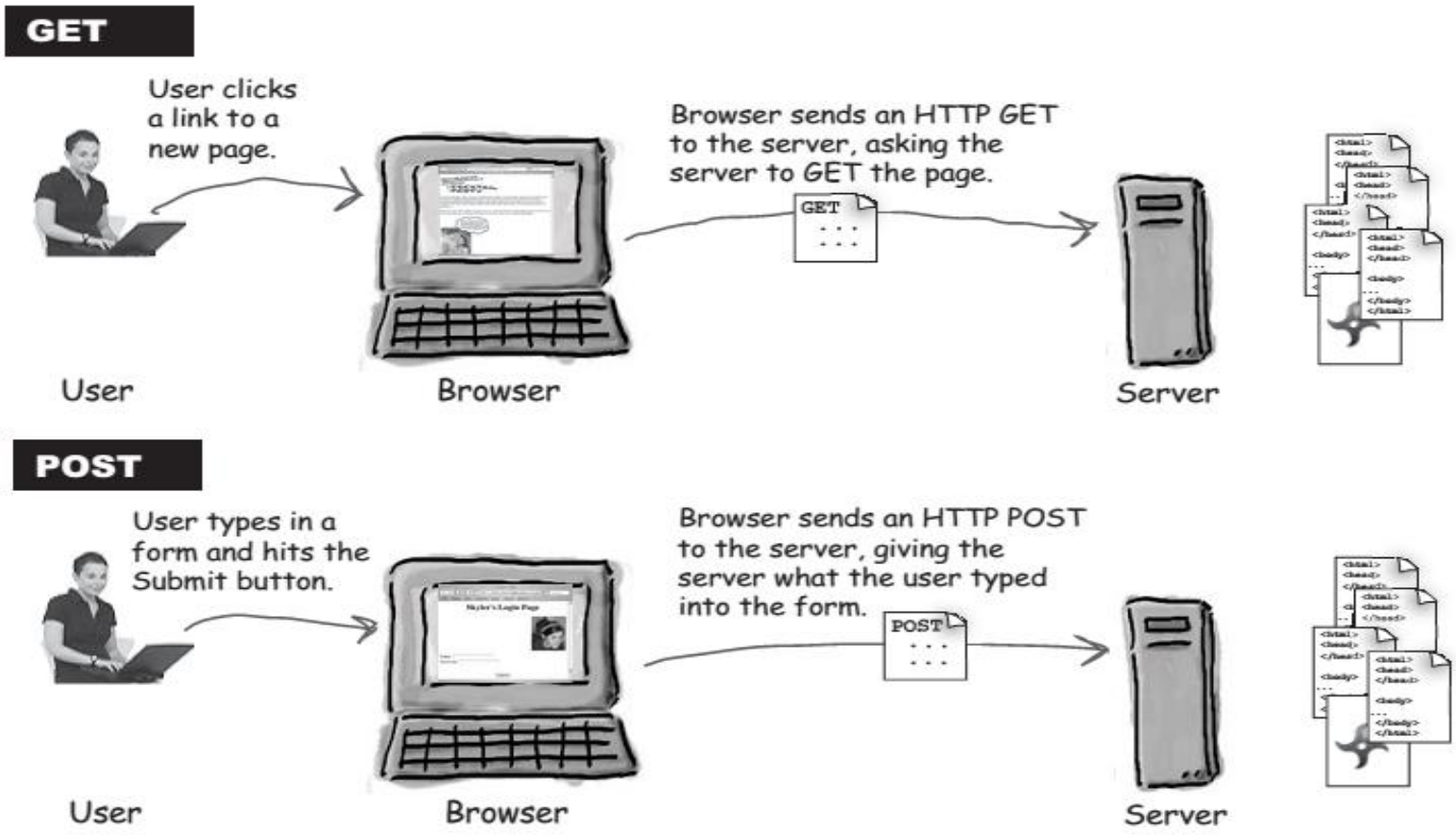
- default in HTTP/1.1
- client sends requests as soon as it encounters a referenced object
- as little as one RTT for all the referenced objects

HTTP request message

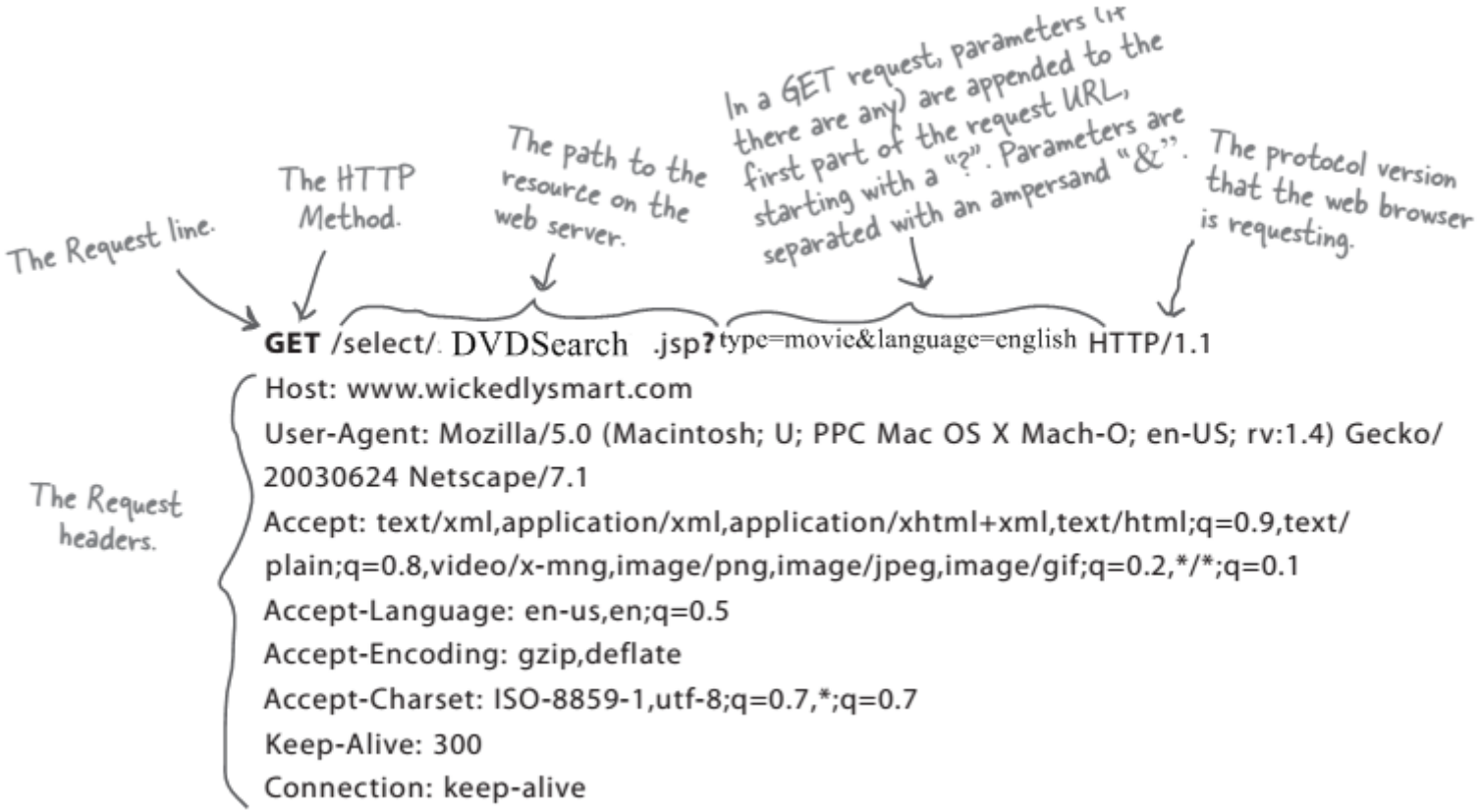
- two types of HTTP messages: *request, response*
- HTTP request message:
  - ASCII (human-readable format)



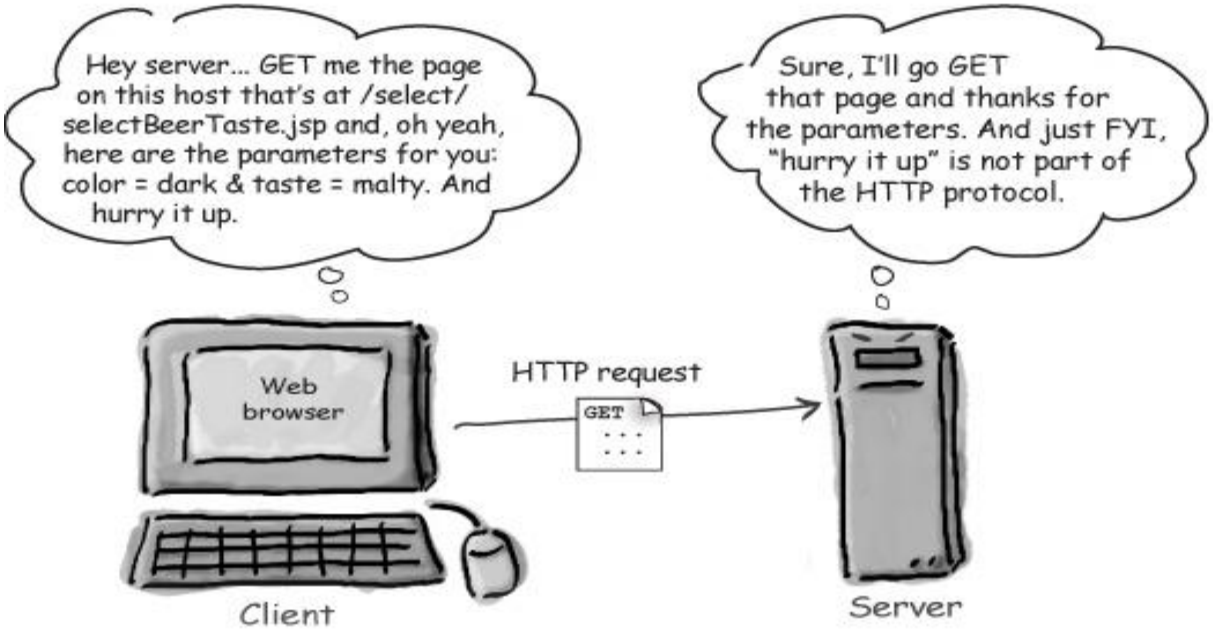
HTTP request message



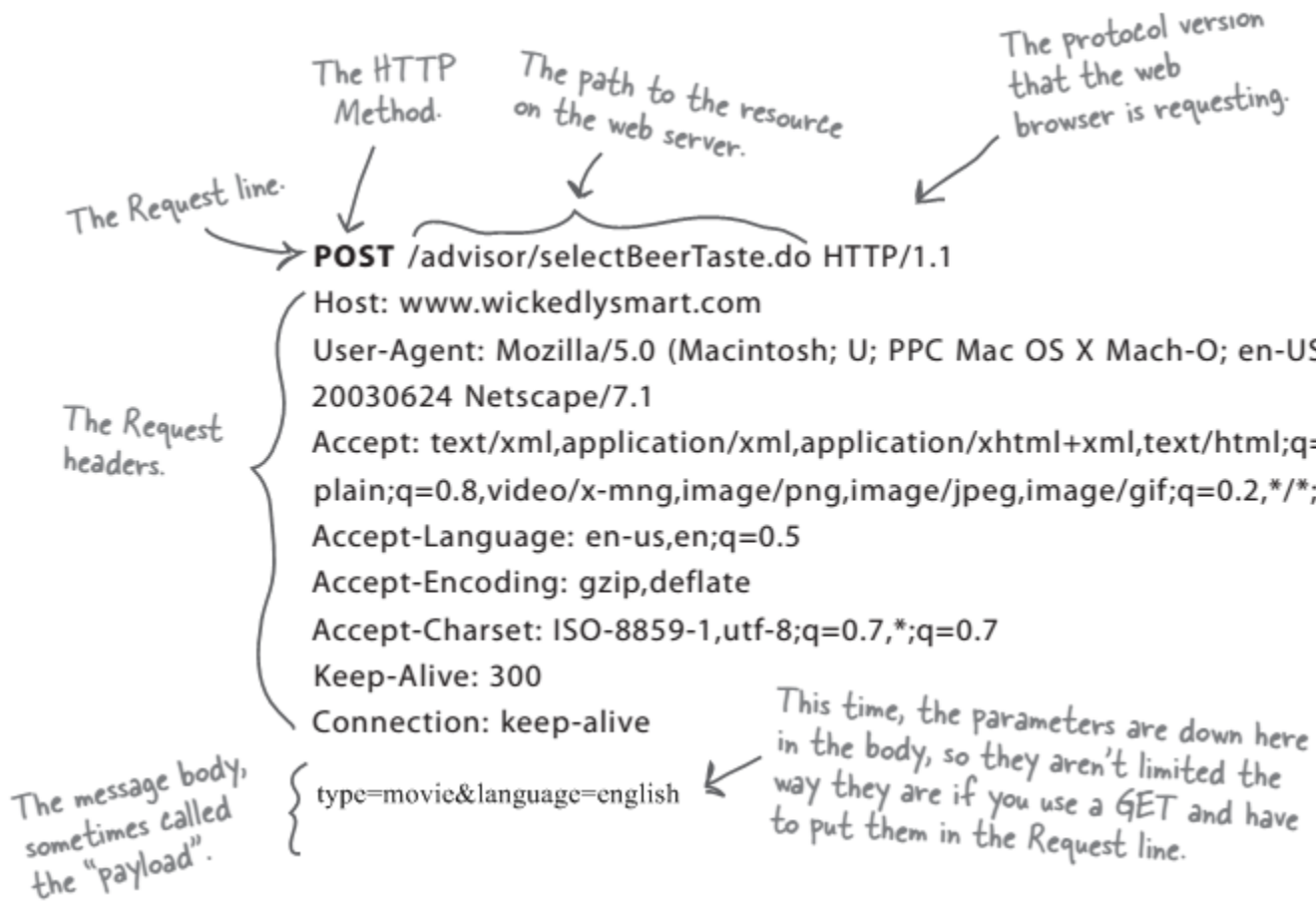
Anatomy of an HTTP GET request



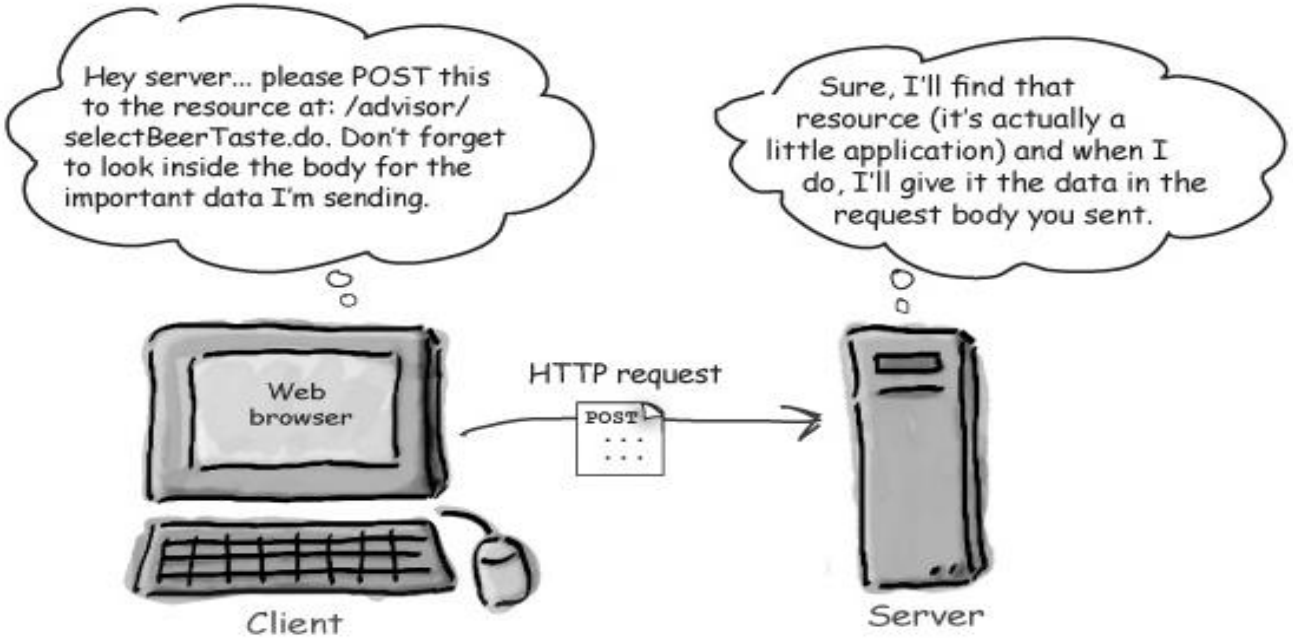
Anatomy of an HTTP GET request



Anatomy of an HTTP POST request



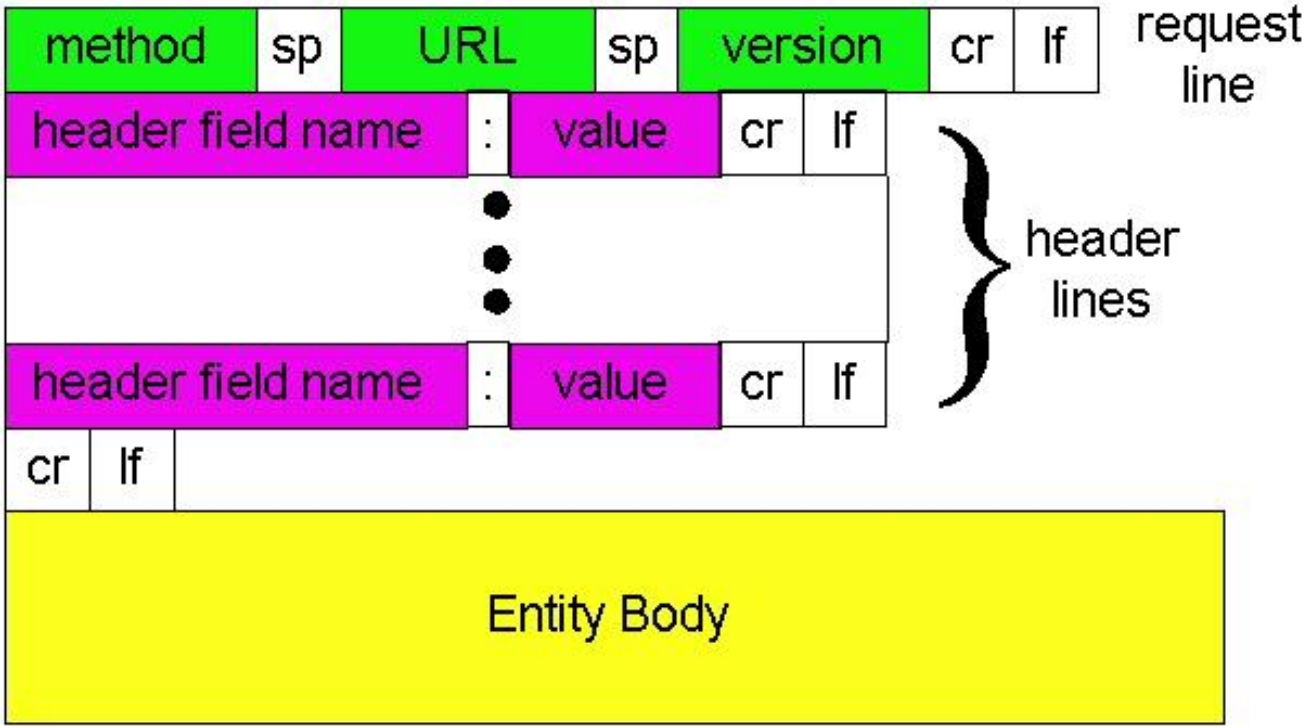
Anatomy of an HTTP POST request



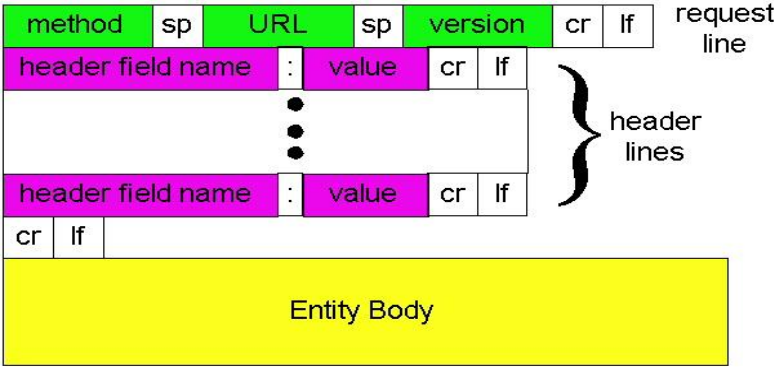
HTTP request message: general format

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
User-agent: Mozilla/4.0
Connection: close
Accept-language:fr
```

(extra carriage return, line feed)



HTTP request message: general format



Now let's look at the header lines in the example. The header line HOST: www.someschool.edu specifies the host on which the object resides. You night think that this header line is unnecessary, as there is already a TCP connection in place to the host. But, as we'll see in Section 2.2.6, the information provided by the host header line is required by Web proxy caches. By including theConnection:close header line, the browser is telling the server that it doesn't want to use persistent connections; it wants the server to close the connection after sending the requested object. Thus the browser that generated this request message implements HTTP/1.1 but it doesn't want to bother with persistent connections. The User-agent: header line specifies the user agent, that is, the browser type that is making the request to the server . Here the user agent is Mozilla/4.0, a Netscape browser. This header line is useful because the server can actually send different versions of the same object to different types of user agents. (Each of the versions is addressed by the same URL.) Finally, the Accept-language: header indicates that the user prefers to receive a French version of the object, if such an object exists on the server; otherwise, the server should send its default version.

The Entity Body is not used with the GET method, but is used with the POST method. The HTTP client uses the POST method when the user fills out a form

Method types

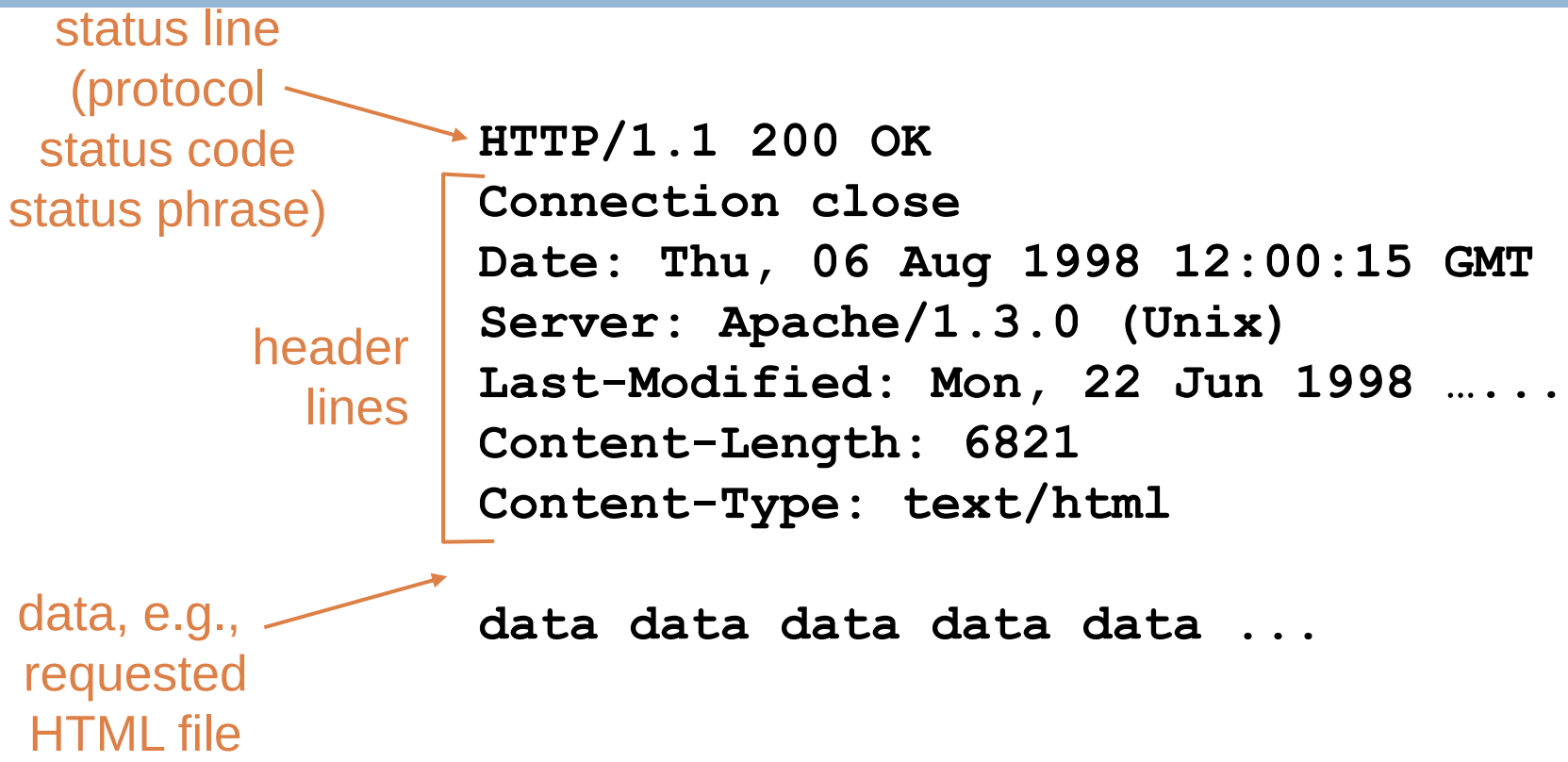
HTTP/1.0

- GET
- POST
- HEAD
  - asks server to leave requested object out of response

HTTP/1.1 **

- GET, POST, HEAD
- PUT
  - uploads file in entity body to path specified in URL field
- DELETE
  - deletes file specified in the URL field

HTTP response message



HTTP response status codes ****

In first line in server->client response message.
A few sample codes:

200 OK

- request succeeded, requested object later in this message

301 Moved Permanently

- requested object moved, new location specified later in this message (Location:)

400 Bad Request

- request message not understood by server

404 Not Found

- requested document not found on this server

505 HTTP Version Not Supported

User-Server Interaction: Authorization and Cookies

- HTTP server is stateless – simplifies server design
- Sometime server needs to identify user
- Two mechanism for identification:

  1. Authorization & 2. CooKies

Authorization :

1) Provide username and password to access documents on server
2) Status code 401: Authorization Required

User-server state: cookies

Many major Web sites use cookies

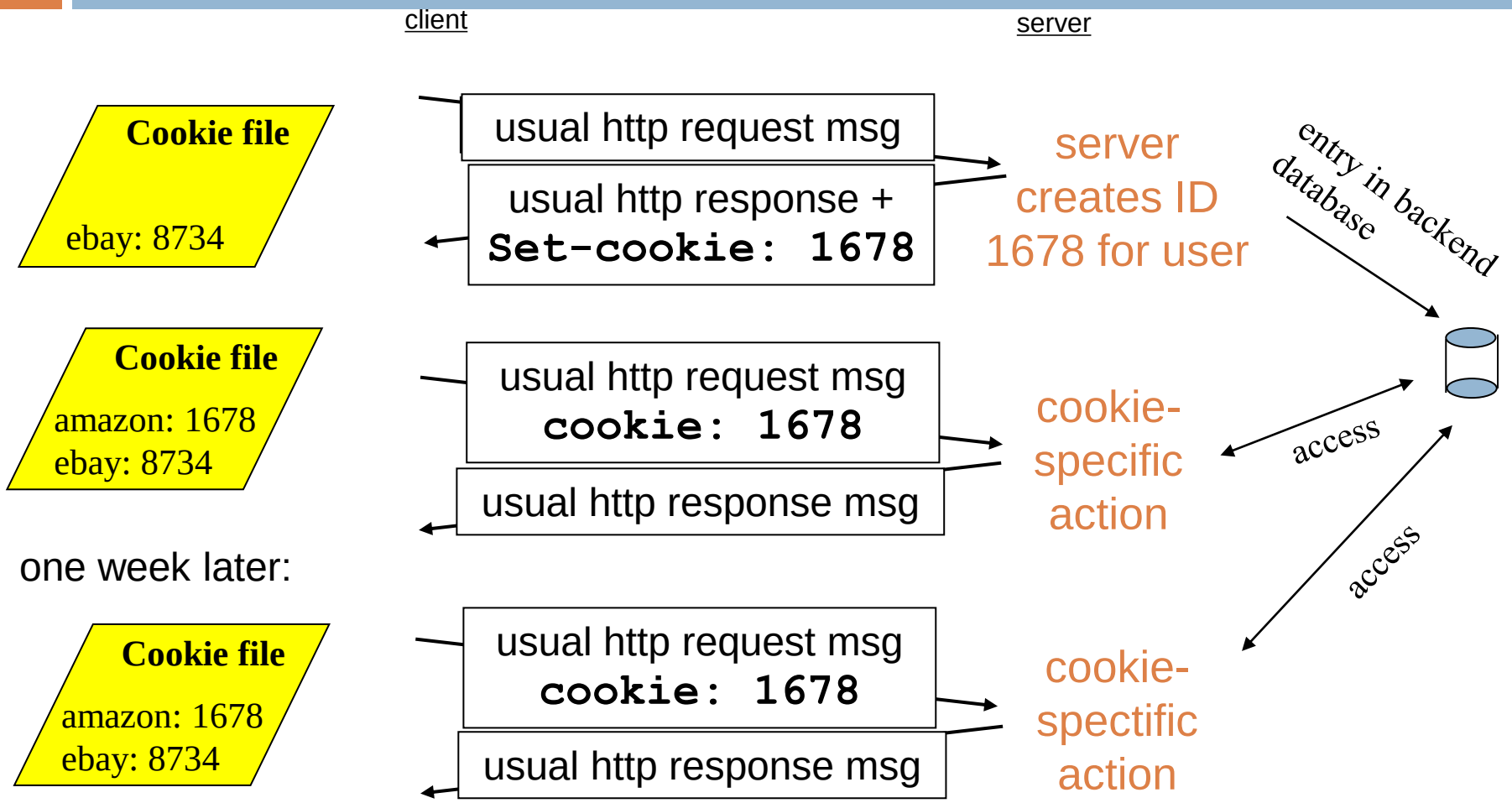
Four components:

1) cookie header line in the HTTP response message
2) cookie header line in HTTP request message
3) cookie file kept on user's host and managed by user's browser
4) back-end database at Web site

Example:

- Susan access Internet always from same PC
- She visits a specific e-commerce site for first time
- When initial HTTP requests arrives at site, site creates a unique ID and creates an entry in backend database for ID

Cookies: keeping "state" (cont.)



Cookies (continued)

What cookies can bring:

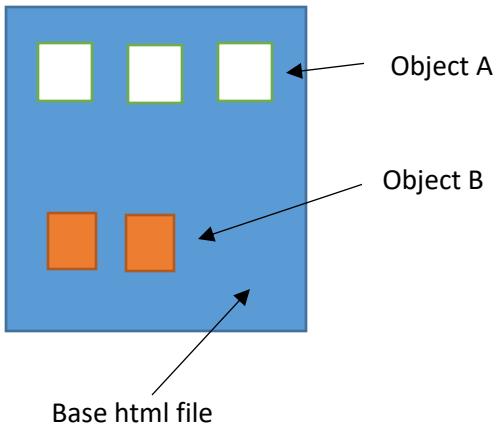
- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

aside

Cookies and privacy:

- cookies permit sites to learn a lot about you
- you may supply name and e-mail to sites
- search engines use redirection & cookies to learn yet more
- advertising companies obtain info across sites

Thank you



Given,

RTT(Round trip time) = 100ms

Transmit time of base html, T_{html} = 5ms

Transmit time of Object A, T_A = 3ms

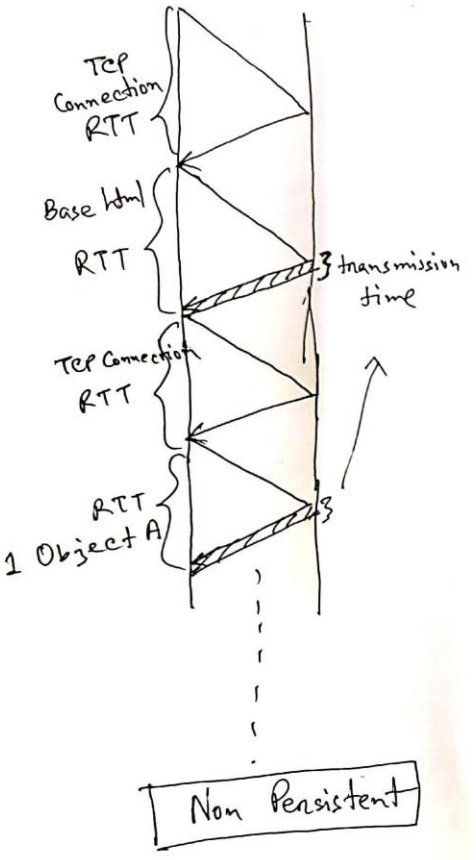
Transmit time of Object B, T_B = 2ms

Number of Object A, N_A = 3

Number of Object B, N_B = 2

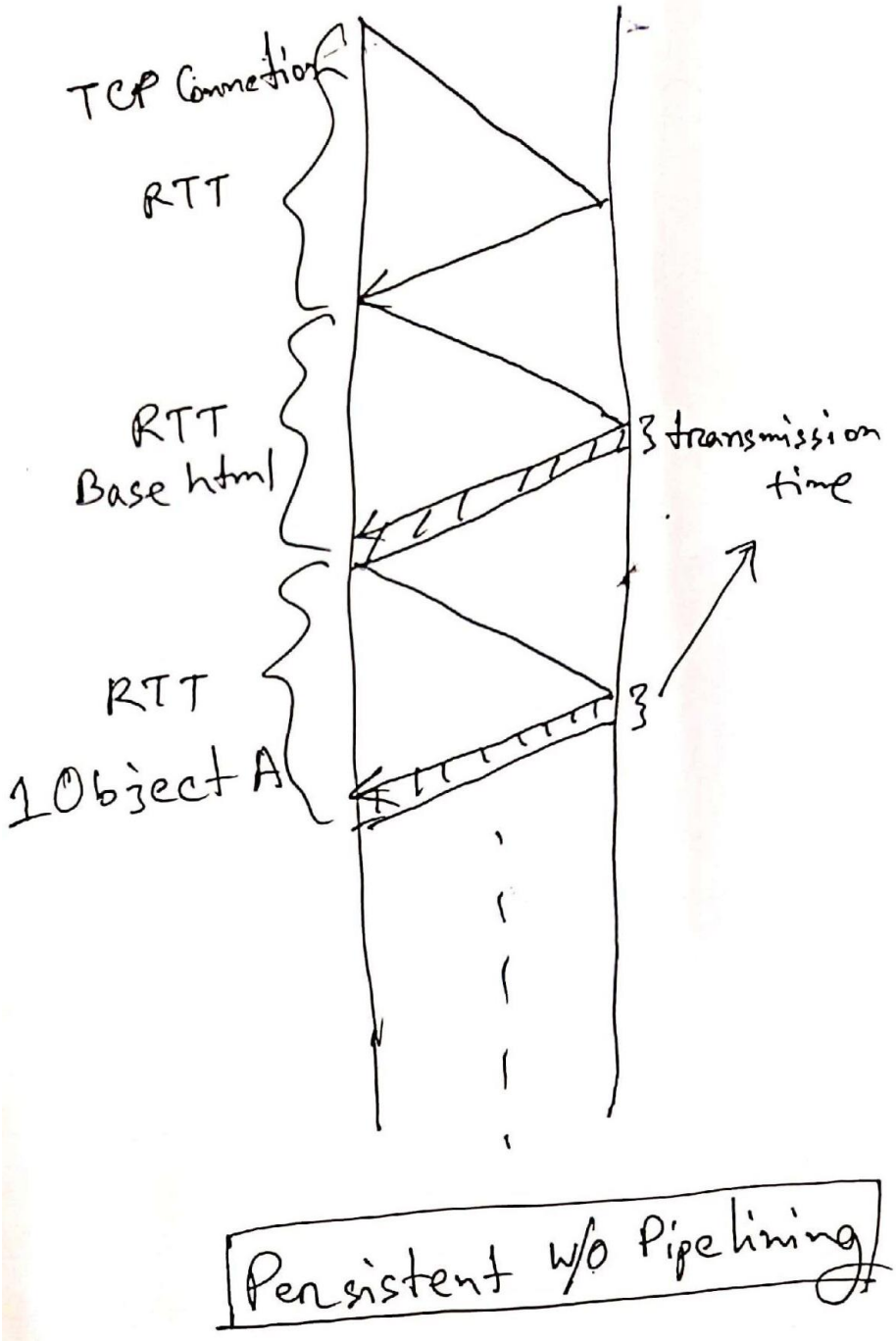
Non Persistent HTTP:

Response time of base html, $R_{html} = 2RTT + T_{html} = 205$ ms
Response time of Object A, $R_A = (2RTT+T_A) \times N_A = 609$ ms
Response time of Object B, $R_B = (2RTT+T_B) \times N_B = 404$ ms
Total response time = $R_{html}+R_A+R_B = 1218$ ms



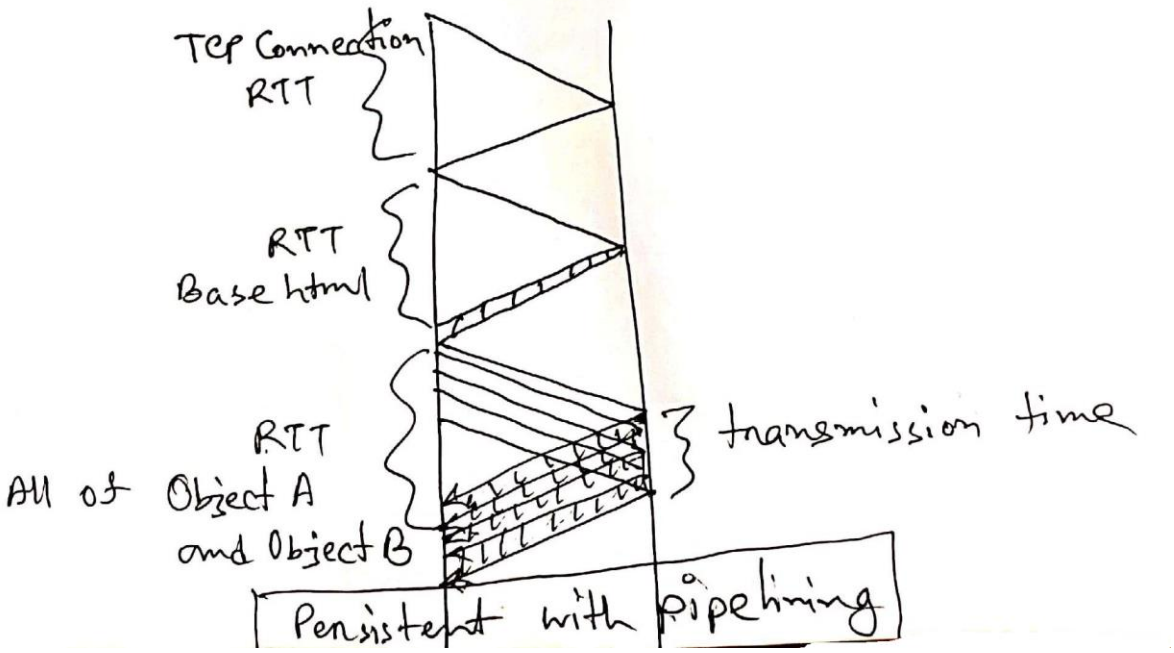
Persistent HTTP without pipelining:

Response time of base html, $R_{html} = 2RTT + T_{html} = 205$ ms
Response time of Object A, $R_A = (RTT+T_A) \times N_A = 309$ ms
Response time of Object B, $R_B = (RTT+T_B) \times N_B = 204$ ms
Total response time = $R_{html}+R_A+R_B = 718$ ms



Persistent HTTP with pipelining:

Response time of base html, $R_{html} = 2RTT + T_{html} = 205$ ms
Response time of Object A and Object B, $R_{AB} = (RTT + T_A \times N_A + T_B \times N_B) = 113$ ms
Total response time = $R_{html} + R_{AB} = 318$ ms



CSE414: WEB ENGINEERING

Daffodil International University

LEARNING OUTCOMES

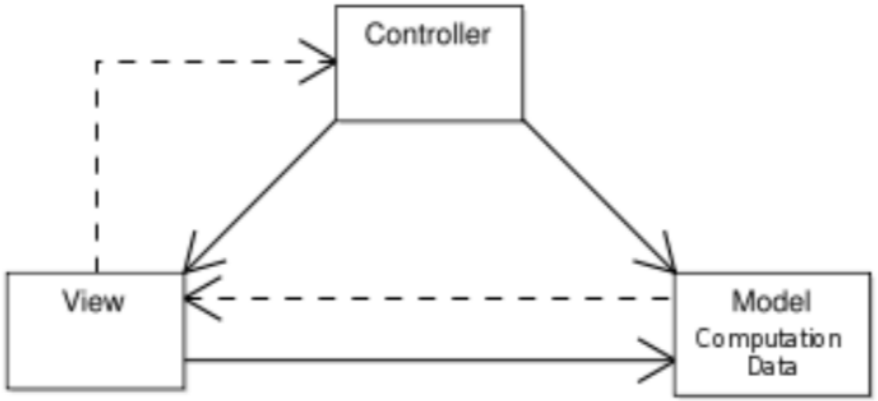
- ✔ You will know MVC design pattern
- ✔ You will be able to apply Project management techniques

DISTRIBUTED SYSTEMS: FUNDAMENTAL QUESTIONS

- Software developers have to consider a wide, but rather stable, range of questions including:
  - Where can or should computations take place?
  - Where can or should data be stored?
  - How fast can data be transferred/communicated?
  - What is the cost of data storage/computations/communication depending on how/where we do it?
  - How robustly/securely can data storage/computations/communication be done depending on how/where we do it?
  - How much energy is available to support data storage/computations/communication depending on how/where we do it?
  - What is the legality of data storage/computations/communications depending on how/where we do it?
- The possible answers to each of these questions is also rather stable, but the 'right' answers change

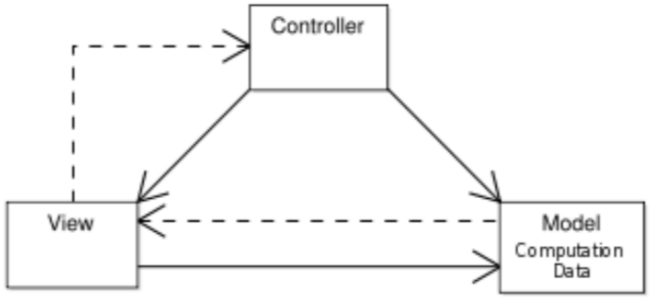
DISTRIBUTED SYSTEMS: MVC(1/3) *****

- We use the Model-View-Controller(MVC) software design pattern to discuss some of these questions in more detail:
  - The model manages the behavior and data
  - The view renders the model into a form suitable for interaction
  - The controller receives user input and translates it into instructions for the model



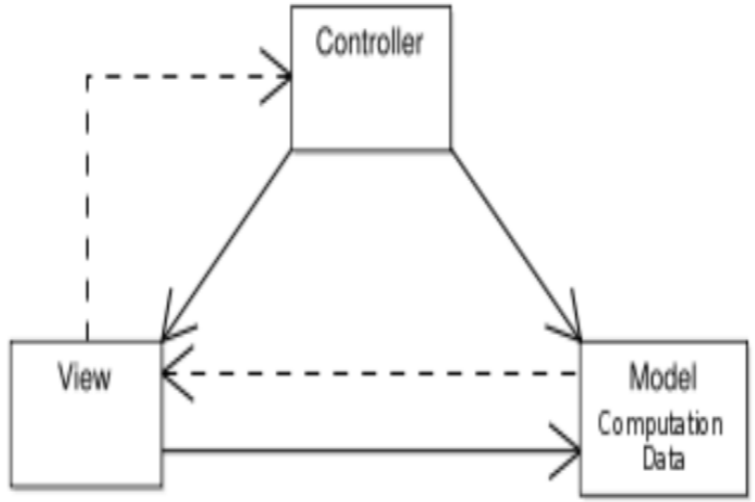
DISTRIBUTED SYSTEMS: MVC(2/3)

- Where should the view be rendered?
  - On the user's computer
  - On a central server (farm) possibly shared by a multitude of users
- Where should the behaviour of the model be computed?
  - Close to the user,
    - on a single computer exclusively used by the user
  - Away from the user,
    - on a central server (farm) shared by a multitude of users
  - Distributed,
    - on several computers owned by a large group of users



DISTRIBUTED SYSTEMS: MVC(3/3)

- Where should the data for the model be held?
  - Close to the user,
    - on a single computer exclusively used by the user
  - Away from the user,
    - on a central server (farm) shared by a multitude of users
  - Distributed,
    - on several computers owned by a large group of users



DISTRIBUTED SYSTEMS: FUNDAMENTAL QUESTIONS

- Software developers have to consider a wide, but rather stable, range of questions
- The possible answers to each of these questions is also rather stable
- The 'right' answer to each these questions will depend on
  - the domain in which the question is posed
  - available technology
  - available resources
- The 'right' answer to each of the questions changes over time
- We may go back and forth between the various answers
- The reasons for that are not purely technological, but includes
  - legal factors
  - social factors
  - economic factors

A BASIC EXAMPLE

In order for a program to get data from a database, it has to undergo a list of actions:

1. Connect to the database server
2. Select a database
3. Query the database
4. Fetch the data
5. Use the Data

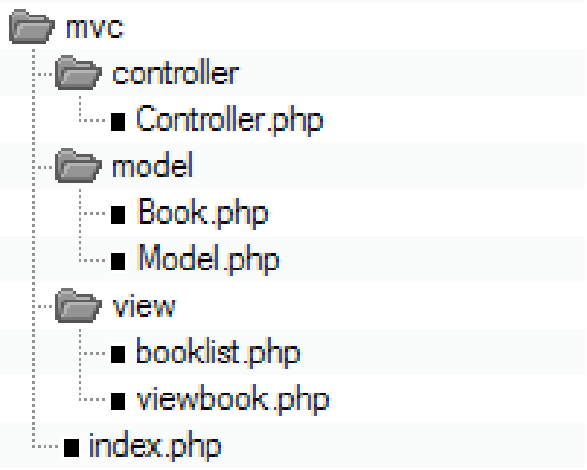
A **framework** may handle steps 1-4 for you, so that your responsibilities are reduced to:

1. Tell the framework to fetch the data
2. Use the data

- There are many frameworks that use MVC design pattern.
  - Laravel, CodeIgniter, Symfony, CakePHP etc. are few of them

EXAMPLE

directory structure



model/Book.php

```php
<?php
    class Book {
        public $title;
        public $author;
        public $description;
        public function __construct($title, $author, $description){
            $this->title = $title;
            $this->author = $author;
            $this->description = $description;
        }
    }
?>
```

EXAMPLE

model/Model.php

```php
<?php
include_once("model/book.php");
class Model {
    public function getBookList(){
        // instead of these values(harcoded),
        //we use SQL queries to output data
        $example_data = array( "Jungle Book" => new Book("Jungle Book", "R. Kipling", "A classic book."),
                               "Moonwalker" => new Book("Moonwalker", "J. Walker", ""),
                                "Harry Potter" => new Book("Harry Potter", "J.K.Rowling", "Fantasy"));
        return $example_data;
    }
    public function getBook($title){
        //We used getBookList() function to manipulate data
        //Again, we use a real database to do this operations here
        $allBooks = $this->getBookList();
        return $allBooks[$title];  }
    }
?>
```

EXAMPLE

view/abook.php

```php
<html>
<head></head>
<body>
    <?php

        echo 'Title:' . $book->title . '<br/>';
        echo 'Author:' . $book->author . '<br/>';
        echo 'Description:' . $book->description . '<br/>';
    ?>
</body>
</html>
```

EXAMPLE

view/booklist.php

```
<html>
<head></head>
<body>
    <table>
        <tr>
            <td>Title</td>
            <td>Author</td>
            <td>Description</td>
        </tr>
        <?php
            foreach ($books as $book){
                echo '<tr>
                        <td><a href="index.php?book='.$book->title.'">'.$book->title .'</a></td>
                        <td>'.$book->author .'</td>
                        <td>'.$book->description.'</td>
                        </tr>';
            }
        ?>
    </table>
</body>
</html>
```

EXAMPLE

controller/Controller.ph

```php
<?php
include_once("model/Model.php");

class Controller {
    public $model;
    public function __construct(){
        $this->model = new Model();
    }

    public function invoke(){
        if (!isset($_GET['book'])){
            //no specific book is requested,
            //show all available books
            $books = $this->model->getBookList();
            include 'view/booklist.php';
        }else{
            //show the requested book
            $book = $this->model->getBook($_GET['book']);
            include 'view/abook.php';
        }
    }
}
?>
```

index.php

```php
<?php
    include_once("controller/Controller.php");
    //interactions start at index
    //forwarded to controller
    $controller = new Controller();
    $controller->invoke();
?>
```

Remember, everything starts at index.php

Apply this basic pattern in your project!

PROJECT MANAGEMENT

- **Deliver on time and on schedule and in accordance with the requirements**
- **Budget and schedule constraints**
- The product is **intangible**
  - Cannot be seen or touched.
  - Project **managers can not see progress by simply looking at the artifact**
- Many software projects are 'one-off' projects
    - Large software projects are usually different from previous projects
    - Experience does not help.
- Software processes are variable and organization specific
    - cannot predict when a process is likely to lead to development

PROJECT MANAGEMENT ACTIVITIES(1/2) *****

- Project planning
  - Project managers are responsible for planning. estimating and scheduling project development and assigning people to tasks.
- Reporting
  - Project managers are usually responsible for reporting on the progress of a project to customers and to the managers of the company developing the software.
- Risk management
  - Project managers assess the risks that may affect a project, monitor these risks and take action when problems arise.

PROJECT MANAGEMENT ACTIVITIES(2/2) *****

- People management
  - Project managers have to choose people for their team and establish ways of working that leads to effective team performance.
- Proposal writing
  - The first stage in a software project may involve writing a proposal to win a contract to carry out an item of work. The proposal describes the objectives of the project and how it will be carried out.

PROJECT MANAGEMENT CHALLENGES

- **Unique software systems:** the experience from the past project is too little to be able to make reliable cost estimates.
- **Extremely technical leadership perspective:** dominated by technology freaks.
- **Poor planning:** many projects are characterize by unclear or incomplete planning objectives, frequent changes to planning objectives, defects in project organization.
- **Development Challenges**
  - Individuality of programmers
  - High number of alternative solutions
  - Rapid technological change
- **Monitoring Challenges**
  - The immaterial state of software products

- **EXERCISE**
  - What are the drawbacks and benefits of using MVC design pattern?
  - Apply MVC while structuring your project
  - Write briefly about obstacles faced in your project
- **READINGS**
  - https://developer.chrome.com/apps/app_frameworks
  - https://www.guru99.com/mvc-tutorial.html

ACKNOWLEDGEMENT

- This module is designed and created with the help from following sources-
  - https://cgi.csc.liv.ac.uk/~ullrich/COMP519/
  - http://www.csc.liv.ac.uk/~martin/teaching/comp519/
  - Materials of MVC, S Rahman Shammi, Daffodil International University

CSE417: WEB ENGINEERING

Daffodil International University

Learning Outcomes

You will learn

- SQL
- To access mySQL database
- To create a basic mySQL database
- To use some basic queries
- To use PHP and mySQL
- Session & Cookie

Introduction to SQL *****

SQL is an ANSI (American National Standards Institute) standard computer language for accessing and manipulating databases.

- SQL stands for Structured Query Language
- using SQL can you can
  - access a database
  - execute queries, and retrieve data
  - insert, delete and update records
- SQL works with database programs like **MS Access, DB2, Informix, MS SQL Server, Oracle, Sybase, mySQL**, etc.

Unfortunately, there are many different versions. But, they must support the same major keywords in a similar manner such as SELECT, UPDATE, DELETE, INSERT, WHERE, etc.

Most of the SQL database programs also have their own proprietary extensions!

The University of Liverpool CS department has a version of mySQL installed on the servers, and it is this system that we use in this course. Most all of the commands discussed here should work with little (or no) change to them on other database systems.

SQL Database Tables

A database most often contains one or more tables. Each table is identified by a name (e.g. "Customers" or "Orders"). Tables contain records (rows) with data.

For example, a table called "Persons":

LastName	FirstName	Address	City
Hansen	Ola	Timoteivn 10	Sandnes
Svendson	Tove	Borgvn 23	Sandnes
Pettersen	Kari	Storgt 20	Stavanger

The table above contains three records (one for each person) and four columns (**LastName, FirstName, Address, and City**).

SQL Queries

With SQL, you can query a database and have a result set returned.

A query like this:

```
SELECT LastName FROM Persons;
```

gives a result set like this:

LastName
Hansen
Svendson
Pettersen

The mySQL database system requires a semicolon at the end of the SQL statement!

Inserting Data

Using `INSERT INTO` you can insert a new row into your table. For example,

```
mysql> INSERT INTO students
    -> VALUES(NULL,'Russell','Martin',396310,'martin@csc.liv.ac.uk');
```

```
Query OK, 1 row affected (0.00 sec)
```

Using `SELECT FROM` you select some data from a table.

```
mysql> SELECT * FROM students;
```

```
+-----+---------+--------+------------+----------------------+
| num | f_name  | l_name | student_id | email                |
+-----+---------+--------+------------+----------------------+
|   1 | Russell | Martin |     396310 | martin@csc.liv.ac.uk |
+-----+---------+--------+------------+----------------------+
1 row in set (0.00 sec)
```

Inserting Some More Data

You can repeat inserting until all data is entered into the table.

```
mysql> INSERT INTO students
    -> VALUES(NULL,'James','Bond',007,'bond@csc.liv.ac.uk');
Query OK, 1 row affected (0.01 sec)


mysql> SELECT * FROM students;
+-----+---------+--------+------------+----------------------+
| num | f_name  | l_name | student_id | email                |
+-----+---------+--------+------------+----------------------+
|   1 | Russell | Martin |     396310 | martin@csc.liv.ac.uk |
|   2 | James   | Bond   |          7 | bond@csc.liv.ac.uk   |
+-----+---------+--------+------------+----------------------+
2 rows in set (0.00 sec)
```

Note: The value “NULL” in the “num” field is automatically replaced by the SQL interpreter as the “auto_increment” option was selected when the table was defined.

Getting Data Out of the Table

- The SELECT command is the main way of getting data out of a table, or set of tables.

```
SELECT * FROM students;
```

Here the asterisk means to select (i.e. return the information in) all columns.

You can specify one or more columns of data that you want, such as

```
SELECT  f_name,l_name FROM students;
```

```
+---------+--------+
| f_name  | l_name |
+---------+--------+
| Russell | Martin |
| James   | Bond   |
+---------+--------+
2 rows in set (0.00 sec)
```

Getting Data Out of the Table (cont.)

- You can specify other information that you want in the query using the **WHERE** clause.

```
SELECT * FROM students WHERE l_name='Bond';
```

```
+-----+---------+--------+------------+---------------------+
| num | f_name  | l_name | student_id | email               |
+-----+---------+--------+------------+---------------------+
|   2 | James   | Bond   |          7 | bond@csc.liv.ac.uk  |
+-----+---------+--------+------------+---------------------+
1 row in set (0.00 sec)
```

```
SELECT student_id, email FROM students WHERE l_name='Bond';
```

```
+------------+---------------------+
| student_id | email               |
+------------+---------------------+
|          7 | bond@csc.liv.ac.uk  |
+------------+---------------------+
1 row in set (0.00 sec)
```

Updating the Table

The UPDATE statement is used to modify data in a table.

```
mysql> UPDATE students SET date='2007-11-15' WHERE num=1;
```

```
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> SELECT * FROM students;
+-----+---------+--------+------------+----------------------+------------+
| num | f_name  | l_name | student_id | email                | date       |
+-----+---------+--------+------------+----------------------+------------+
|   1 | Russell | Martin |     396310 | martin@csc.liv.ac.uk | 2007-11-15 |
|   2 | James   | Bond   |          7 | bond@csc.liv.ac.uk   | NULL       |
+-----+---------+--------+------------+----------------------+------------+
2 rows in set (0.00 sec)
```

Note that the default date format is “YYYY-MM-DD” and I don’t believe this default setting can be changed.

Deleting Some Data

The DELETE statement is used to delete rows in a table.

```
mysql> DELETE FROM students WHERE l_name='Bond';
```

```
Query OK, 1 row affected (0.00 sec)
```

```
mysql> SELECT * FROM students;
+-----+---------+--------+------------+----------------------+------------+
| num | f_name  | l_name | student_id | email                | date       |
+-----+---------+--------+------------+----------------------+------------+
|   1 | Russell | Martin |     396310 | martin@csc.liv.ac.uk | 2006-11-15 |
+-----+---------+--------+------------+----------------------+------------+
1 row in set (0.00 sec)
```

Simulation - The Final Table

We'll first add another column, update the (only) record, then insert more data.

```
mysql> ALTER TABLE students ADD gr INT;
Query OK, 1 row affected (0.01 sec)
Records: 1  Duplicates: 0  Warnings: 0
mysql> SELECT * FROM students;
+-----+---------+--------+------------+----------------------+------------+------+
| num | f_name  | l_name | student_id | email                | date       | gr   |
+-----+---------+--------+------------+----------------------+------------+------+
|   1 | Russell | Martin |     396310 | martin@csc.liv.ac.uk | 2007-11-15 | NULL |
+-----+---------+--------+------------+----------------------+------------+------+
1 row in set (0.00 sec)
mysql> UPDATE students SET gr=3 WHERE num=1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
mysql> SELECT * FROM students;
+-----+---------+--------+------------+----------------------+------------+------+
| num | f_name  | l_name | student_id | email                | date       | gr   |
+-----+---------+--------+------------+----------------------+------------+------+
|   1 | Russell | Martin |     396310 | martin@csc.liv.ac.uk | 2007-11-15 |    3 |
+-----+---------+--------+------------+----------------------+------------+------+
1 row in set (0.00 sec)

mysql> INSERT INTO students VALUES(NULL,'James','Bond',007,'bond@csc.liv.ac.uk','2007-
11-15', 1);
. . .
. . .
```

Simulation - The Final Table (cont.)

```
. . .
. . .
mysql> INSERT INTO students VALUES(NULL,'Hugh','Milner',75849789,'hugh@poughkeepsie.ny',
    CURRENT_DATE, 2);
```

Note: **CURRENT_DATE** is a built-in SQL command which (as expected) gives the current (local) date.

```
mysql> SELECT * FROM students;
+-----+---------+----------+------------+---------------------------+------------+------+
| num | f_name  | l_name   | student_id | email                     | date       | gr   |
+-----+---------+----------+------------+---------------------------+------------+------+
|   1 | Russell | Martin   |     396310 | martin@csc.liv.ac.uk      | 2007-11-15 |    3 |
|   5 | Kate    | Ash      |     124309 | kate@ozymandius.co.uk     | 2007-11-16 |    3 |
|   3 | James   | Bond     |          7 | bond@csc.liv.ac.uk        | 2007-11-15 |    1|
|   4 | Bob     | Jones    |      12190 | bob@nowhere.com           | 2007-11-16 |    3 |
|   6 | Pete    | Lofton   |         76 | lofton@iwannabesedated.com | 2007-11-17 |    2 |
|   7 | Polly   | Crackers |       1717 | crackers@polly.org        | 2007-11-17 |    1|
|   8 | Hugh    | Milner   |   75849789 | hugh@poughkeepsie.ny      | 2007-11-17 |    2 |
+-----+---------+----------+------------+---------------------------+------------+------+
7 rows in set (0.00 sec)
mysql> exit
Bye
```

PHP: MySQL Database [CRUD Operations]

To do any operations, you need to Connect to your database first!

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";
// Create connection
$conn = mysqli_connect($servername, $username, $password,
$dbname);
// Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
?>
```

Inserting Data

```
 //Remember, you always connect to your database first
 //Assumption: We have a Table named MyGuests


$sql = "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('John', 'Doe', 'john@example.com')";

if (mysqli_query($conn, $sql)) {
    echo "New record created successfully";
} else {
    echo "Error: " . $sql . "<br>" . mysqli_error($conn);
}
```

Reading Data

```
//Remember, you always connect to your database first
//Assumption: We have a Table named MyGuests


$sql = "SELECT id, firstname, lastname FROM MyGuests";
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    // output data of each row
    while($row = $result->fetch_assoc()) {
        echo "id: " . $row["id"]. " - Name: " .
$row["firstname"]. " " . $row["lastname"]. "<br>";
}
} else {
    echo "0 results";
}
```

More..

- Did you close your connection?

```
mysqli_close($conn);
//put this at the end of operation
```

- Similarly you can update and delete data
-
- More here : PHP: MySQL Database

- There are slides hidden and supposed to help you doing more complex opertions
- These will not be covered in Class Lecture.
- Most of them are already explained to you in DATABASE MANAGEMENT SYSTEM course

Attention!

- There is easier way to do using PHPMyadmin accesses via localhost (after XAMPP installation)
- Remember, you will not always get flexibilty and,
- You love command line tools to write code!
- Cheers!

Exercise

- Show one example of each of CRUD operation
- Do two complex operations
- **READINGS/Practice**
  - M Schafer: Ch. 31
  - W3 schools
  - http://www.php-mysql-tutorial.com/
  - http://www.sitepoint.com/php-security-blunders/
  - http://php.net/manual/en/mysqli.quickstart.prepared-statements.php

Acknowledgement

- This module is designed and created with the help from following sources-
  - https://cgi.csc.liv.ac.uk/~ullrich/COMP519/
  - http://www.csc.liv.ac.uk/~martin/teaching/comp519/

Putting Content into Your Database with PHP

We can simply use PHP functions and mySQL queries together:

- Connect to the database server and login (this is the PHP command to do so)

```
mysql_connect("host","username","password");
```

- Choose the database

```
mysql_select_db("database");
```

Host:	mysql
Database:	martin
Username:	martin
Password:	<blank>

- Send SQL queries to the server to add, delete, and modify data

```
mysql_query("query");
```

(use the exact same query string as you would normally use in SQL, without the trailing semi-colon)

- Close the connection to the database server (to ensure the information is stored properly)

```
mysql_close();
```

Student Database: data_in.php

```
<html>
<head>
<title>Putting Data in the DB</title>
</head>
<body>
<?php
/*insert students into DB*/
if(isset($_POST["submit"]))  {

   $db = mysql_connect("mysql", "martin");
   mysql_select_db("martin");

   $date=date("Y-m-d");  /*  Get the current date in the right SQL format  */

   $sql="INSERT INTO students  VALUES(NULL,'" . $_POST["f_name"] . "','" .
   $_POST["l_name"] . "'," . $_POST["student_id"] . ",'" . $_POST["email"] .
   "','" . $date . "'," . $_POST["gr"] . ")";      /*  construct the query  */

   mysql_query($sql);   /*  execute the query  */
   mysql_close();

   echo"<h3>Thank you. The data has been entered.</h3> \n";

   echo'<p><a href="data_in.php">Back to registration</a></p>' . "\n";

   echo'<p><a href="data_out.php">View the student lists</a></p>' ."\n";

   }
```

Student Database: data_in.php

```
else  {
?>
<h3>Enter your items into the database</h3>
<form action="data_in.php" method="POST">
First Name: <input type="text" name="f_name" /> <br/>
Last Name: <input type="text" name="l_name" /> <br/>
ID: <input type="text" name="student_id" /> <br/>
email: <input type="text" name="email" /> <br/>
Group: <select name="gr">
       <option value ="1">1</option>
       <option value ="2">2</option>
       <option value ="3">3</option>
</select><br/><br/>
<input type="submit" name="submit" /> <input type="reset" />
</form>

<?php
   }
?>

</body>
</html>
```

view the output page

Getting Content out of Your Database with PHP

Similarly, we can get some information from a database:

- Connect to the server and login, choose a database

```
mysql_connect("host","username","password");
mysql_select_db("database");
```

  - Send an SQL query to the server to select data from the database into an array

```
$result=mysql_query("query");
```

  - Either, look into a row and a fieldname

```
$num=mysql_numrows($result);
$variable=mysql_result($result,$i,"fieldname");
```

    - Or, fetch rows one by one

```
$row=mysql_fetch_array($result);
```

- Close the connection to the database server

```
mysql_close();
```

Student Database: data_out.php

```
<html>
<head>
<title>Getting Data out of the DB</title>
</head>
<body>
<h1> Student Database </h1>
<p> Order the full list of students by
<a href="data_out.php?order=date">date</a>,
<href="data_out.php?order=student_id">id</a>, or
by <a href="data_out.php?order=l_name">surname</a>.
</p>

<p>
<form action="data_out.php" method="POST">
Or only see the list of students in group
<select name="gr">
  <option value ="1">1</option>
  <option value ="2">2</option>
  <option value ="3">3</option>
</select>
<br/>
<input type="submit" name="submit‟ />
</form>
</p>
```

Student Database: data_out.php

```
<?php
/*get students from the DB */
$db = mysql_connect("mysql",“martin");
mysql_select_db(“martin", $db);

switch($_GET["order"]){
case  'date':   $sql = "SELECT * FROM students ORDER BY date"; break;
case  ‘student_id':     $sql = "SELECT * FROM students ORDER BY student_id";
   break;
case  ‘l_name': $sql = "SELECT * FROM students ORDER BY l_name"; break;

default: $sql = “SELECT * FROM students”;  break;
}
if(isset($_POST["submit"])){
  $sql = “SELECT * FROM students WHERE gr=“ . $_POST["gr"];
}

$result=mysql_query($sql);      /*  execute the query  */
while($row=mysql_fetch_array($result)){
  echo "<h4> Name: “ . $row["l_name"] . ', ‘ . $row["f_name"] . "</h4> \n";
  echo "<h5> ID: “ . $row[“student_id"] . "<br/> Email: “ . $row["email"] .
   "<br/> Group: “ . $row["gr"] . "<br/> Posted: “ . $row["date"] . "</h5> \n";
}
mysql_close();
?>
</body>
</html>
```

view the output page

Using several tables

- mySQL (like any other database system that uses SQL) is a relational database, meaning that it's designed to work with multiple tables, and it allows you to make queries that involve several tables.
- Using multiple tables allows us to store lots of information without much duplication.
- Allows for easier updating (both insertion and deletion).
- We can also perform different types of queries, combining the information in different ways depending upon our needs.

Advanced Queries

- We can link these tables together by queries of this type:

```
myql> SELECT * from clients, purchases WHERE clients.client_id=purchases.client_id ORDER BY purchase_id;
+----------+---------+--------+------------------------+---------------+----------+------------+-----------+------------+
|client_id | f_name  | l_name | address                | city          | postcode | purchase_id | client_id | date       |
----------+---------+--------+------------------------+---------------+----------+------------+-----------+------------+
|        1 | Russell | Martin | Dept of Computer Science | Liverpool     | L69 3BX  |          1 |         1 | 2007-11-09 |
|        1 | Russell | Martin | Dept of Computer Science | Liverpool     | L69 3BX  |          2 |         1 | 2007-11-10 |
|        2 | Bob     | Milnor | 12 Peachtree Ln        | Liverpool     | L12 3DX  |          4 |         2 | 2007-11-20 |
|        4 | Larry   | Vance  | 76 Jarhead Ln          | Liverpool     | L12 4RT  |          5 |         4 | 2007-11-20 |
|        3 | Sarah   | Ford   | 542b Jersey Rd         | West Kirby    | L43 8JK  |          6 |         3 | 2007-11-21 |
|        5 | Paul    | Abbott | 90 Crabtree Pl         | Leamingotn Spa | CV32 7YP |          7 |         5 | 2007-11-25 |
|        3 | Sarah   | Ford   | 542b Jersey Rd         | West Kirby    | L43 8JK  |          8 |         3 | 2007-11-25 |
+----------+---------+--------+------------------------+---------------+----------+------------+-----------+------------+
7 rows in set (0.01 sec)

mysql>
```

So you can see that this query basically gives us all of the purchase orders that have been placed by the clients (but not the number of items, or the items themselves).

More Complex Queries

- We can create most any type of query that you might think of with a (more complicated) "WHERE" clause:

```
mysql> SELECT purchases.purchase_id, f_name, l_name, date
   FROM purchases, clients WHERE
   purchases.client_id=clients.client_id;
+-------------+---------+--------+------------+
| purchase_id | f_name  | l_name | date       |
+-------------+---------+--------+------------+
|           1 | Russell | Martin | 2007-11-09 |
|           2 | Russell | Martin | 2007-11-10 |
|           4 | Bob     | Milnor | 2007-11-20 |
|           5 | Larry   | Vance  | 2007-11-20 |
|           6 | Sarah   | Ford   | 2007-11-21 |
|           7 | Paul    | Abbott | 2007-11-25 |
|           8 | Sarah   | Ford   | 2007-11-25 |
+-------------+---------+--------+------------+
7 rows in set (0.00 sec)

mysql>
```

More Complex Queries (cont.)

- Find the purchases by the person named “Ford”.

```
mysql> SELECT purchases.purchase_id, f_name, l_name, date FROM
   purchases, clients
       WHERE (purchases.client_id=clients.client_id) AND
   (l_name='Ford');
+-------------+--------+--------+------------+
| purchase_id | f_name | l_name | date       |
+-------------+--------+--------+------------+
|           6 | Sarah  | Ford   | 2007-11-21 |
|           8 | Sarah  | Ford   | 2007-11-25 |
+-------------+--------+--------+------------+
2 rows in set (0.01 sec)

mysql>
```

Cookie Workings

`setcookie(name,value,expire,path,domain)` **creates cookies.**

```
<?php
setcookie("uname", $_POST["name"], time()+36000);
?>
<html>
<body>
<p>
Dear <?php echo $_POST["name"] ?>, a cookie was set on this
page! The cookie will be active when the client has sent the
cookie back to the server.
</p>
</body>
</html>
```

NOTE:
setcookie() **must appear BEFORE** <html> **(or any output) as it's part of the header information sent with the page.**

```
<html>
<body>
<?php
if ( isset($_COOKIE["uname"]) )
echo "Welcome " . $_COOKIE["uname"] . "!<br
/>";
else
echo "You are not logged in!<br />";
?>
</body>
</html>
```

`$_COOKIE`
contains all COOKIE data.

`isset()`
finds out if a cookie is set

use the cookie name as a variable

Session

- When you work with an application, you open it, do some changes, and then you close it. This is much like a Session.
  - HTTP address doesn't maintain state.
  - Session variables solve this problem by storing user information to be used across multiple pages (e.g. username, favorite color, etc).
  - By default, session variables last until the user closes the browser.
  - Session variables hold information about one single user
  - available to all pages in one application. ie, logged in

Example

```php
<?php
// Start the session
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// Set session variables
$_SESSION["favcolor"] = "green";
$_SESSION["favanimal"] = "cat";
echo "Session variables are set.";
?>

</body>
</html>
```

Exercise

- Design a form and validate its data.
- Design a user registration system
- **READINGS/Practice**
  - M Schafer: Ch. 29-32
  - https://www.w3schools.com/php/php_form_validation.asp
  - Designing a Sign-up/Log-in page

Acknowledgement

- This module is designed and created with the help from following sources-
  - https://cgi.csc.liv.ac.uk/~ullrich/COMP519/
  - http://www.csc.liv.ac.uk/~martin/teaching/comp519/
  -

Microservices: Principles, Tradeoffs, Tradeoffs, and Adoption

Nafiz Ahmed Emon
Lecturer
Department of CSE

CONTENTS

01

Definition & Core Characteristics

Definition & Core Characteristics

What

• Autonomous services modeled around business domains.

Key Traits

***** 4 traits

• Independent Deployability (No coordination needed).

• Own State (Private databases, polyglot persistence).

• Size ("Small enough to rebuild in 2-4 weeks").

• Alignment with Org Structure (Conway’s Law).

02

Monoliths – Types & Challenges

Monoliths – Types & Challenges

***** three type

Single-Process Monolith

- Pros: Simple to deploy.
- Cons: Scales poorly.

Modular Monolith

- Pros: Cleaner structure.
- Cons: Still monolithic.

Distributed Monolith

- Anti-pattern: Coupled services, shared failures.

03

Microservices Advantages

Microservices Advantages (Detailed)

Advantage	Explanation
Technology Heterogeneity	Use Python for ML, Java for transactions.
Robustness	Isolated failures (e.g., Inventory Service down ≠ Checkout down).
Scaling	Scale "Checkout Service" during Black Friday, not the entire app.
Organizational Alignment	Teams own full lifecycle (DevOps culture).

04

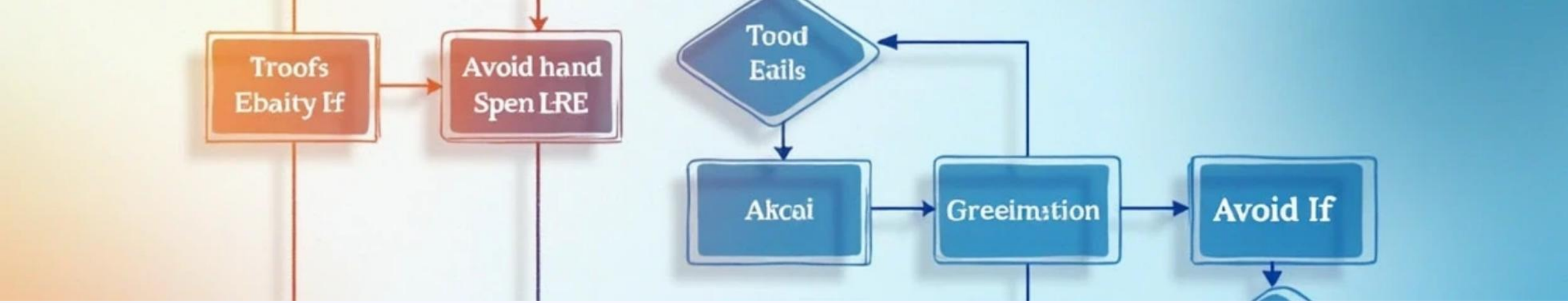
Pain Points & Mitigations

Pain Points & Mitigations *****

Challenge	Solution
Data Consistency	Eventual consistency (Sagas, CQRS).
Monitoring	Distributed tracing (Jaeger, OpenTelemetry).
Testing	Contract testing (Pact), chaos engineering.
Cost	Cloud cost optimization (auto-scaling, serverless).

05

When to Adopt? Decision Framework



When to Adopt? Decision Framework

1

☑ Use Microservices If:

- Multiple teams working on the same system.
- Parts of the system have different scaling needs.
- Rapid feature delivery is critical.

2

✕ Avoid If:

- Your team is small (< 5 engineers).
- You lack CI/CD and observability tools.
- Strong transactions (e.g., banking systems).\

06

Enabling Technologies

Enabling Technologies

Containers/Kubernetes (Isolation + orchestration).

Streaming (Kafka for event-driven communication).

Serverless (AWS Lambda for ephemeral tasks).

Log Aggregation (ELK Stack for debugging).

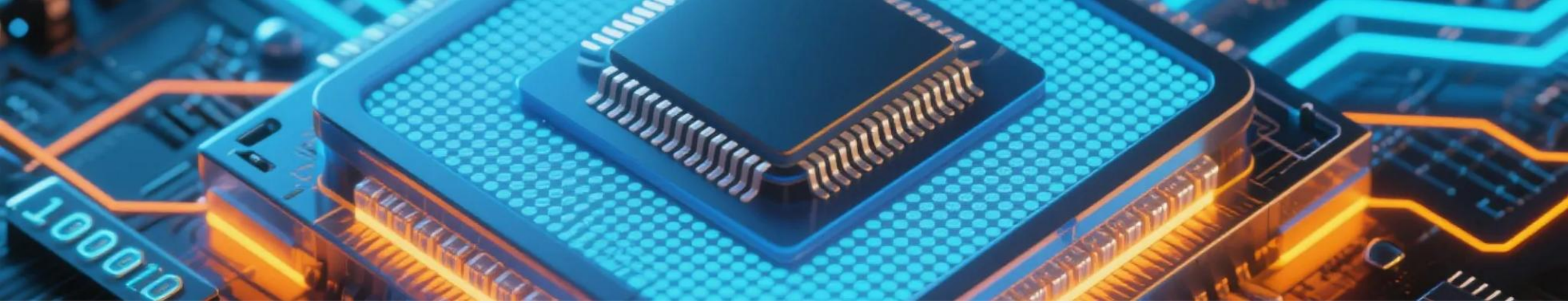
07

Summary & Key Takeaways

Summary & Key Takeaways

1. Microservices enable scalability and speed but add complexity.
2. Not a silver bullet – Assess team size, domain complexity, and tooling.
3. Monoliths are fine until scaling demands separation.

Thank You



Designing Microservices: Boundaries, Boundaries, Coupling, and DDD

Nafiz Ahmed Emon
Lecturer
Department of CSE

CONTENTS

01

Service Boundaries – Core Principles

Service Boundaries – Core Principles ****

- High Cohesion (Related functions stay together).
- Loose Coupling (Minimize dependencies).
- Information Hiding (Encapsulate internals).

02

Coupling Types (Ranked Best → Worst)

*****

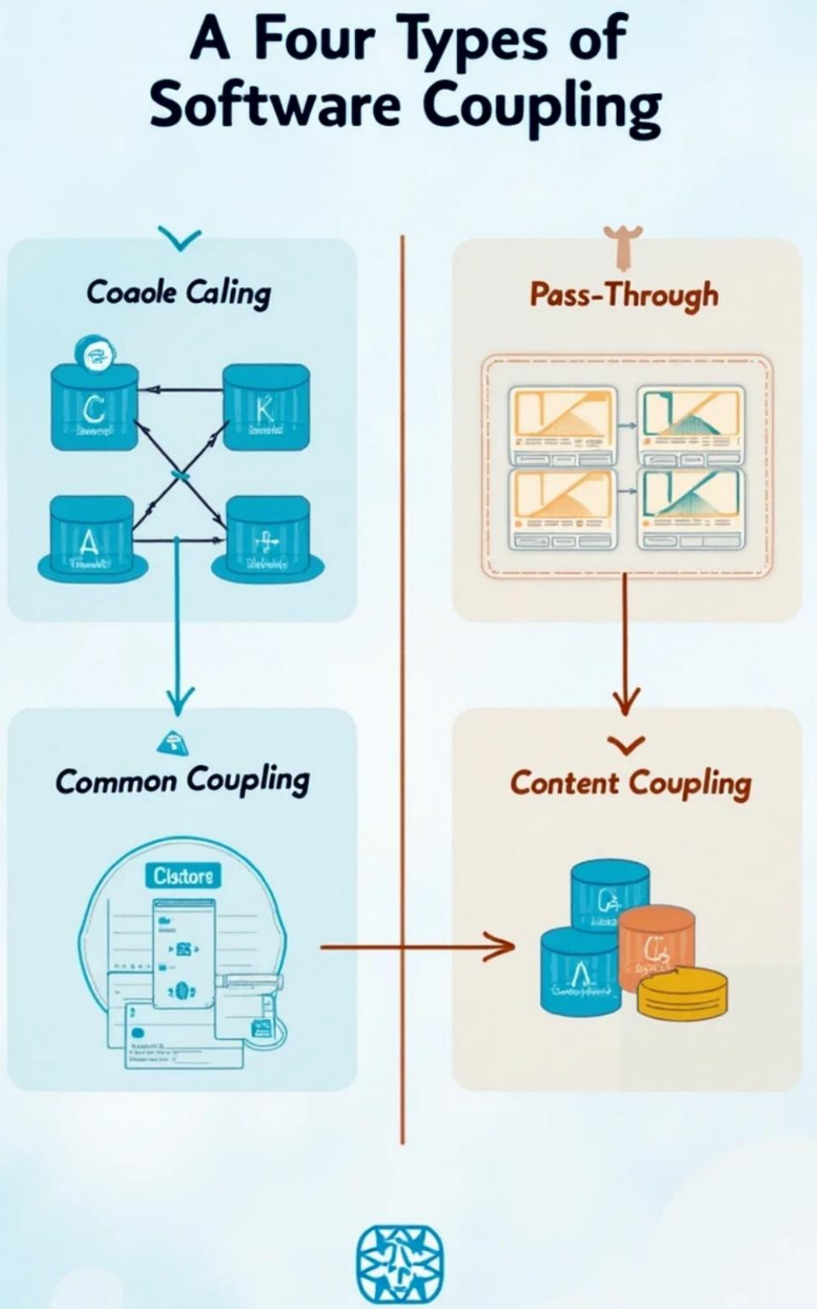
Coupling Types (Ranked Best → → Worst)

Domain Coupling (Good): Services exchange domain events (e.g., OrderPlaced).

Common Coupling (Worse): Shared databases/libs.

Pass-Through Coupling (Bad): Middleman calls (A → B → C).

Content Coupling (Worst): Direct DB access between services.



03

Domain-Driven Design (DDD) Essentials

*

Domain-Driven Design (DDD) Essentials

Ubiquitous Language (Shared glossary for devs/business).

Aggregates (Transactional units, e.g., Order + OrderItems).

Bounded Contexts (Clear ownership, e.g., "Shipping" vs. "Inventory").

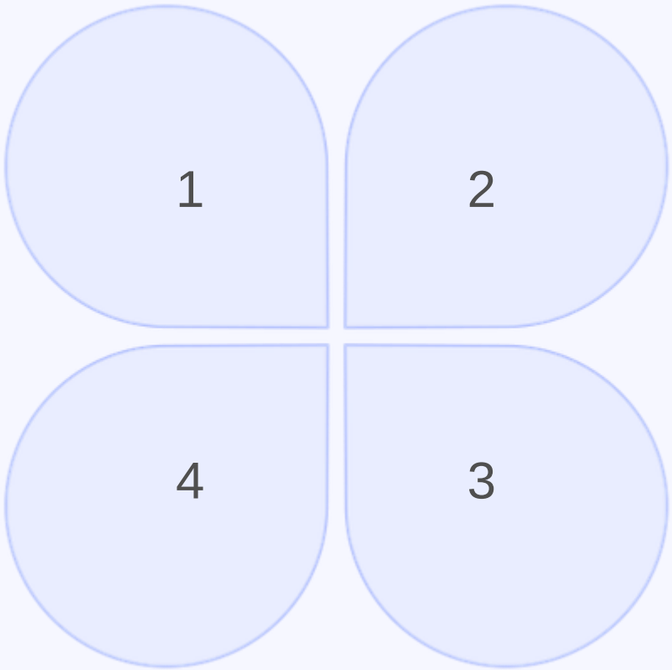
04

Event Storming Workshop

Event Storming Workshop

Step-by-Step:

Domain Events (Sticky notes: "OrderPlaced", "PaymentProcessed").



Commands (Actions triggering events: "PlaceOrder").

Bounded Contexts (Draw service boundaries).

Aggregates (Who handles commands? "OrderAggregate").

05

Alternative Boundary Strategies

Alternative Boundary Strategies

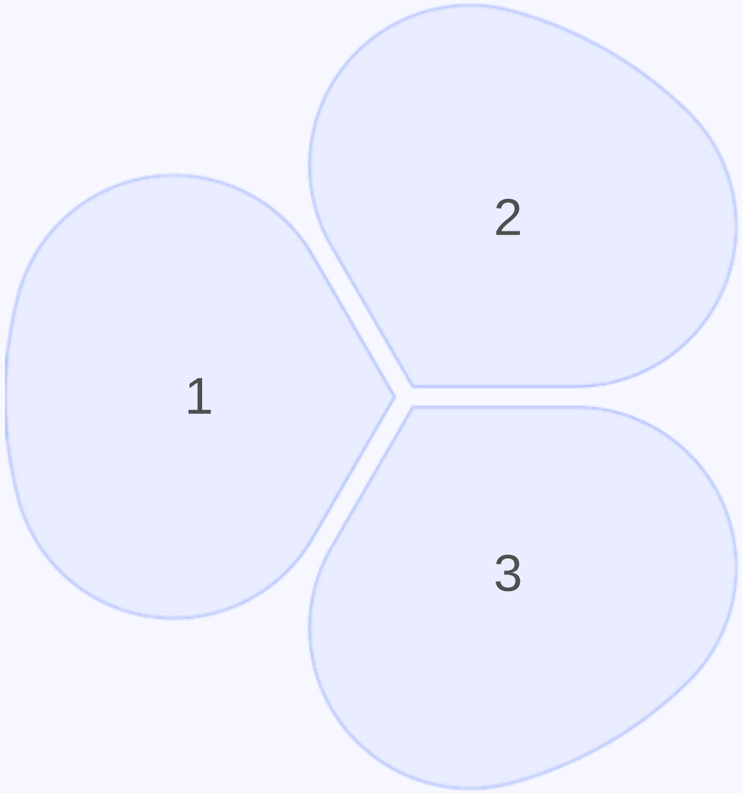
Strategy	Use Case	Example
Volatility	Frequent changes	A/B testing service.
Data	Reporting needs	Analytics microservice.
Technology	Specialized tools (GPU, ML)	Image recognition service.

06

Anti-Patterns & Pitfalls

Anti-Patterns & Pitfalls

Distributed Monolith (Tightly coupled "microservices").



Chatty Services (Excessive API calls → latency).

Shared Databases (Breaks encapsulation).

07

Case Study – MusicCorp Example

Case Study – MusicCorp Example Example

Problem

Monolithic music store.

Solution

- Bounded Contexts: Catalog, Orders, Inventory.
- Events: OrderPlaced → InventoryUpdated.

08

Summary & Actionable Insights

Summary & Actionable Insights

1

Start with DDD to find natural boundaries.

2

Minimize coupling – Prefer events over direct calls.

3

Workshop it – Use Event Storming with stakeholders.

Thank You